

```
import os
import time
import requests
import threading
import random
import re
from collections import deque
from flask import Flask, jsonify, request
from datetime import datetime, timezone,
timedelta
import pandas as pd
import numpy as np

app = Flask(__name__)

# =====
# TÜRKİYE SAATİ (UTC+3) – sunucu saati ne
# olursa olsun bu kullanılır
# =====
TR_TZ = timezone(timedelta(hours=3))

def now_tr():
    """Sunucunun saat dilimi farklı olsa
    bile her zaman Türkiye saatini (UTC+3)
    döndürür."""
    return
    datetime.now(timezone.utc).astimezone(TR_TZ
    )

TELEGRAM_TOKEN =
```

```
os.environ.get("TELEGRAM_TOKEN")
CHAT_ID = os.environ.get("CHAT_ID")
TD_API_KEY = os.environ.get("TD_API_KEY")
ANTHROPIC_API_KEY =
os.environ.get("ANTHROPIC_API_KEY")
FINNHUB_API_KEY =
os.environ.get("FINNHUB_API_KEY")

# ===== 6 KANAL YAPISI =====
# Her kategori için ayrı Telegram
kanalı/grubu. Henüz ayarlanmamışsa (env
# variable yoksa) eski tek CHAT_ID'ye düşer
– kanalları teker teker
# ekleyebilmen için geriye dönük uyumlu.
# ===== TEK GRUP + 6 KONU (TOPIC) YAPISI
=====
# Sercan ayrı ayrı grup yerine tek bir
supergroup içinde Telegram'ın
# "Konular" (Forum Topics) özelliğini
kullanmayı tercih etti. Bu yüzden
# chat_id hepsinde AYNI, ayrım
"message_thread_id" ile yapılıyor.
# Henüz thread ID'ler belirlenmediyse (env
variable yoksa) o kategori de
# grubun Genel (ana) konusuna düşer.
HAWK_GROUP_CHAT_ID =
os.environ.get("HAWK_GROUP_CHAT_ID",
CHAT_ID)
THREAD_ID_TREND =
os.environ.get("THREAD_ID_TREND")
THREAD_ID_ERKENUYARI =
os.environ.get("THREAD_ID_ERKENUYARI")
THREAD_ID_FORMASYON =
os.environ.get("THREAD_ID_FORMASYON")
```

```

THREAD_ID_ELLIOTT =
os.environ.get("THREAD_ID_ELLIOTT")
THREAD_ID_HABER =
os.environ.get("THREAD_ID_HABER")
THREAD_ID_SISTEM =
os.environ.get("THREAD_ID_SISTEM")

_KANAL_THREAD_MAP = {
    "trend": THREAD_ID_TREND,
    "erkenuyari": THREAD_ID_ERKENUYARI,
    "formasyon": THREAD_ID_FORMASYON,
    "elliott": THREAD_ID_ELLIOTT,
    "haber": THREAD_ID_HABER,
    "sistem": THREAD_ID_SISTEM,
}

def send_kanal(message, kanal_key):
    """
        kanal_key: 'trend' | 'erkenuyari' |
        'formasyon' | 'elliott' | 'haber' |
        'sistem'
        Aynı gruba, ilgili konunun (topic)
        thread_id'siyle mesaj gönderir.
    """
    send_telegram(message,
HAWK_GROUP_CHAT_ID,
_KANAL_THREAD_MAP.get(kanal_key))

# ===== HABER MÜKERRER GÖNDERİM ÖNLEME
=====
# Gönderilen her haberin kimliğini (Finnhub
news id) tutar, aynı haber
# hem sinyal tetiklemesinden hem periyodik
taramadan iki kez gelmesin diye.

```

```
_SENT_NEWS_MAXLEN = 1500
sent_news_queue =
deque(maxlen=_SENT_NEWS_MAXLEN)
sent_news_set = set()

def is_news_already_sent(news_id):
    return news_id in sent_news_set

def mark_news_sent(news_id):
    if news_id in sent_news_set:
        return
    if len(sent_news_queue) ==
sent_news_queue.maxlen:
        oldest = sent_news_queue[0]
        sent_news_set.discard(oldest)
    sent_news_queue.append(news_id)
    sent_news_set.add(news_id)

# tradingview-scraper kütüphanesi
(opsiyonel - yoksa stockanalysis.com
fallback kullanılır)
try:
    from
tradingview_scraper.symbols.market_movers
import MarketMovers
    from tradingview_scraper.symbols.news
import NewsScraper
    from
tradingview_scraper.symbols.overview import
Overview
    TV_SCRAPER_AVAILABLE = True
except Exception:
    TV_SCRAPER_AVAILABLE = False
```

```
# =====
# SEKTÖR HARİTASI (sinyal mesajında
gösterilir)
# =====

# =====

# Sektör bilgisi artık DİNAMİK çekiliyor
(sabit liste yok).
# Aynı gün içinde tekrar sorgulamamak için
basit bir bellek cache kullanılır.
sector_cache = {}
sma200_cache = {} # {"SYMBOL": {"value":
float, "date": "DD.MM.YYYY"}} - günlük
cache, Twelve Data/Barchart kredisini korur

def get_sector(symbol, exchange='NASDAQ'):
    """
        stockanalysis.com/stocks/{symbol}/
sayfasındaki şirket bilgi tablosundan
        Sector bilgisini çeker (gerekirse
Industry'ye düşer).

        NOT: Önceki sürüm tradingview-scraper'a
(TV_SCRAPER_AVAILABLE) bağlıydı,
        bu kütüphane Railway'de hiç aktif
olmadığı için fonksiyon her zaman
        "Bilinmiyor" döndürüyordu. Bu sürüm
OHLCV için kullandığımız aynı
        güvenli yöntemle (önce yorumları
temizle, sonra basit regex ile eşleştir)
        gerçek veri çeker.
    """
    cache_key = symbol.upper()
```

```

        if cache_key in sector_cache:
            return sector_cache[cache_key]

        try:
            headers = {"User-Agent":
"Mozilla/5.0 (Windows NT 10.0; Win64;
x64)"}
            url =
f"https://stockanalysis.com/stocks/{symbol.
lower()}/"
            r = requests.get(url,
headers=headers, timeout=10)
            if r.status_code == 200:
                clean_html = re.sub(r'<!--.*?--
>', '', r.text, flags=re.DOTALL)
                sector_match =
re.search(r'Sector</span>\s*<a[^\>]*>([^\<]+)
</a>', clean_html)
                industry_match =
re.search(r'Industry</span>\s*<a[^\>]*>
([^\<]+)</a>', clean_html)
                sector = (

sector_match.group(1).strip() if
sector_match
                else
industry_match.group(1).strip() if
industry_match
                else "Bilinmiyor"
                )
                sector_cache[cache_key] =
sector
                return sector
        except:

```

pass

```
sector_cache[cache_key] = "Bilinmiyor"  
return "Bilinmiyor"
```

```
def get_sma200_barchart(symbol):  
    """
```

barchart.com/stocks/quotes/{symbol}/technical-analysis sayfasındaki
"Moving Average" tablosundan 200-Day
satırının gerçek SMA200 fiyat
değerini çeker.

NEDEN GEREKLİ: stockanalysis.com'un
/history/ sayfası sayfalamayı
desteklemiyor (her zaman aynı son ~50
günü döndürüyor, ?p=2 parametresi
görmezden geliniyor – Railway
konsolunda doğrulandı). Bu yüzden SMA200
gibi 200 günlük göstergeler
stockanalysis.com'dan hesaplanamıyordu.
Barchart.com'un Technical Analysis
sayfası SMA200'ü HAZIR HESAPLANMIŞ
olarak veriyor, bu yüzden 200 günlük
ham veri çekmemize hiç gerek yok.

GÜNLÜK CACHE: SMA200 gün içinde pratik
olarak değişmez (200 günlük
ortalamada 1 günün ağırlığı ~%0.5). Bu
yüzden değer günde sadece bir
kez çekilip cache'lenir – hem
Barchart'a hem Twelve Data'ya gereksiz
istek atılmasını önler.

```

"""
    cache_key = symbol.upper()
    today_str =
now_tr().strftime('%d.%m.%Y')

    if cache_key in sma200_cache and
sma200_cache[cache_key]["date"] ==
today_str:
        return sma200_cache[cache_key]
["value"]

    try:
        headers = {"User-Agent":
"Mozilla/5.0 (Windows NT 10.0; Win64;
x64)"}
        url =
f"https://www.barchart.com/stocks/quotes/{s
ymbol.upper()}/technical-analysis"
        r = requests.get(url,
headers=headers, timeout=12)
        if r.status_code == 200:
            clean_html = re.sub(r'<!--.*?--
>', '', r.text, flags=re.DOTALL)
            match = re.search(r'<td>200-
Day</td>\s*<td[^>]*>\s*([\d.]+)\s*</td>',
clean_html)
            if match:
                value =
float(match.group(1))
                sma200_cache[cache_key] =
{"value": value, "date": today_str}
                return value
    except:
        pass

```



```

        # Başarısız olursa, eski (dünkü) cache
        değeri varsa onu döndür - hiç yoktan iyidir
        if cache_key in sma200_cache:
            return sma200_cache[cache_key]
["value"]
        return None

# =====
# BELLEK
# =====

portfolio = {}
alerts = {}
tracked = {}
known_chats = {} # /debug/chats için:
{chat_id: {"title":, "type":, "last_seen":,
"last_text":}}
session_sector_signals = {}
news_archive = {}
news_counter = {}

def reset_sector_tracking():
    session_sector_signals.clear()

def register_sector_signal(symbol):
    sector = get_sector(symbol)
    if sector not in
session_sector_signals:
        session_sector_signals[sector] = []

session_sector_signals[sector].append(symbol)
    return sector,

```

```

len(session_sector_signals[sector])

def get_news_id(date_str=None):
    if not date_str:
        date_str =
now_tr().strftime('%d%m%y')
    if date_str not in news_counter:
        news_counter[date_str] = 0
    news_counter[date_str] += 1
    return f"
{date_str}/{news_counter[date_str]}"

# =====
# TELEGRAM
# =====

def send_telegram(message, chat_id=None,
thread_id=None):
    cid = chat_id or CHAT_ID
    url =
f"https://api.telegram.org/bot{TELEGRAM_TOK
EN}/sendMessage"
    payload = {"chat_id": cid, "text":
message, "parse_mode": "HTML"}
    if thread_id:
        try:
            payload["message_thread_id"] =
int(thread_id)
        except (TypeError, ValueError):
            pass
    try:
        r = requests.post(url,
json=payload, timeout=10)
        if r.status_code != 200:

```

```
        print(f"[DEBUG send_telegram]
BAŞARISIZ status={r.status_code} body=
{r.text[:300]}")
    except Exception as e:
        print(f"[DEBUG send_telegram]
EXCEPTION: {e}")

# =====
# CANLI HAVUZ SİSTEMİ
# =====
# Öncelik 1: tradingview-scraper
(MarketMovers) - en güvenilir
# Öncelik 2: stockanalysis.com web scraping
- yedek kaynak
# Öncelik 3: son başarılı canlı taramanın
hafızası - acil durum

last_pool_source = {"source": None, "time":
None, "count": 0}

# Bot manuel olarak /kapat ile
kapatılmışsa, piyasa saatleri ne olursa
olsun
# otomatik tarama durur. /ac ile tekrar
açılır. Varsayılan: açık.
bot_manual_state = {"active": True}
last_successful_pool = [] # En son
başarılı CANLI taramanın sonucu (sabit
liste DEĞİL)

def fetch_pool_tradingview():
    """tradingview-scraper kütüphanesi ile
    gainers+losers+active çeker.
    Sadece NASDAQ ve NYSE hisseleri alınır,
```

OTC/AMEX gibi düşük likiditeli borsalar
elenir."""

```
    if not TV_SCRAPER_AVAILABLE:
        return None
    try:
        symbols = set()
        mm = MarketMovers()
        for category in ['gainers',
'losers', 'most-active']:
            try:
                result =
mm.scrape(market='stocks-usa',
category=category, limit=30)
                if result and
result.get('data'):
                    for item in
result['data']:
                        sym =
item.get('symbol', '')
                        if not
(sym.startswith('NASDAQ:') or
sym.startswith('NYSE:')):
                            continue
                        clean =
sym.split(':')[ -1]
                        if clean and
re.match(r'^[A-Z]{1,5}$', clean):

symbols.add(clean)
            except:
                continue
        return list(symbols) if
len(symbols) >= 15 else None
    except:
```

```

        return None

def fetch_pool_stockanalysis():
    """stockanalysis.com web scraping ile
    gainers/active/losers çeker (yedek
    kaynak)."""
    symbols = set()
    pages = [

        "https://stockanalysis.com/markets/gainers/",
        ",

        "https://stockanalysis.com/markets/active/"
        ,

        "https://stockanalysis.com/markets/losers/"
        ]
    headers = {"User-Agent": "Mozilla/5.0
    (Windows NT 10.0; Win64; x64)"}
    try:
        for url in pages:
            r = requests.get(url,
            headers=headers, timeout=10)
            if r.status_code != 200:
                continue
            found =
            re.findall(r'/stocks/([a-z0-9-]+)/',
            r.text)
            for s in found:
                clean = s.upper()
                if re.match(r'^[A-Z]
                {1,5}$', clean):
                    symbols.add(clean)
    return list(symbols) if

```

```

len(symbols) >= 15 else None
    except:
        return None

def fetch_top_gainers_table(limit=50):
    """
        stockanalysis.com/markets/gainers/
        sayfasından günün en çok kazanan
        hisselerini SIRALI olarak, kazanç
        oranı/fiyat/hacim detaylarıyla çeker.
        Dönen liste: [{"symbol", "price",
        "change_pct", "volume", "dollar_volume"}]
    """
    try:
        headers = {"User-Agent":
        "Mozilla/5.0 (Windows NT 10.0; Win64;
        x64)"}
        r =
        requests.get("https://stockanalysis.com/markets/gainers/", headers=headers,
        timeout=12)
        if r.status_code != 200:
            return []
        clean_html = re.sub(r'<!--.*?-->',
        '', r.text, flags=re.DOTALL)

        # stockanalysis gainers tablosu:
        Sembol | Şirket | Fiyat | Değişim |
        Değişim% | Hacim
        row_pattern = re.compile(
            r'<a href="/stocks/([a-z0-9-]+)"/>["^"]*class=["^"]*ticker["^"]*["^"]*>
            ([A-Z0-9\-\-]+)</a>'
            r'.*?<td["^"]*>([\d.]+)</td>'

```

```

# fiyat
r'.*?<td[^>]*>[^<]*</td>'
# deęişim ($) atla
r'.*?<td[^>]*>\+?([\d.]+)%
</td>' # deęişim%
r'.*?<td[^>]*>([\d.]+[BMK]?)
</td>', # hacim
re.DOTALL
)
matches =
row_pattern.findall(clean_html)

results = []
for slug, symbol, price_str,
change_pct_str, volume_str in matches:
    if not re.match(r'^[A-Z]
{1,5}$', symbol):
        continue
    try:
        price = float(price_str)
        change_pct =
float(change_pct_str)

        # Hacim metnini sayıya
çevir (B=milyar, M=milyon, K=bin)
        vol_text =
volume_str.replace(',', '')
        if vol_text.endswith('B'):
            volume =
float(vol_text[:-1]) * 1_000_000_000
        elif
vol_text.endswith('M'):
            volume =
float(vol_text[:-1]) * 1_000_000

```

```

        elif
vol_text.endswith('K'):
            volume =
float(vol_text[:-1]) * 1_000
        else:
            volume =
float(vol_text)

        dollar_volume = volume *
price

        results.append({
            "symbol": symbol,
            "price": price,
            "change_pct":
change_pct,
            "volume": int(volume),
            "dollar_volume":
dollar_volume,
        })
    except:
        continue
    if len(results) >= limit:
        break

    return results
except:
    return []

def fetch_losers_ranked(limit=100):
    """
    stockanalysis.com/markets/losers/
    sayfasından, EN ÇOK KAYBEDENDEN
    BAŞLAYARAK SIRALI şekilde sembol

```



```
listesi döndürür (sıralama önemli,  
    fetch_pool_stockanalysis'teki gibi  
    karışık set değil).  
    Elliott Dalga otomatik taraması bu  
    sıralı listeyi kullanır.  
    """  
    headers = {"User-Agent": "Mozilla/5.0  
(Windows NT 10.0; Win64; x64)"}  
    symbols = []  
    try:  
        for page in range(1, 4): # her  
            sayfa ~25-50 sembol, 3 sayfa yeterli olmalı  
            url =  
                "https://stockanalysis.com/markets/losers/"  
            params = {"p": page} if page >  
1 else {}  
            r = requests.get(url,  
headers=headers, params=params, timeout=10)  
            if r.status_code != 200:  
                break  
            # Sayfadaki sembol sırasını  
            koru (set kullanmıyoruz)  
            found =  
                re.findall(r'/stocks/([a-z0-9-]+)/',  
r.text)  
            for s in found:  
                clean = s.upper()  
                if re.match(r'^[A-Z]  
{1,5}$', clean) and clean not in symbols:  
                    symbols.append(clean)  
            if len(symbols) >= limit:  
                break  
            return symbols[:limit] if  
len(symbols) >= 15 else None
```

```
except:
    return None

def get_raw_trading_pool():
    """
    Canlı havuzu ÖRNEKLEMESİZ, TAM ham
    liste olarak döner.
    Haber taraması gibi geniş kapsam
    gerektiren işler için kullanılır.
    Aynı üç kademeli kaynak sırasını izler:
    tradingview -> stockanalysis -> hafıza.
    """
    global last_successful_pool

    pool = fetch_pool_tradingview()
    if pool:
        last_pool_source["source"] =
"tradingview"
        last_pool_source["time"] = now_tr()
        last_pool_source["count"] =
len(pool)
        last_successful_pool = pool
        return pool

    pool = fetch_pool_stockanalysis()
    if pool:
        last_pool_source["source"] =
"stockanalysis"
        last_pool_source["time"] = now_tr()
        last_pool_source["count"] =
len(pool)
        last_successful_pool = pool
        return pool
```

```

        if last_successful_pool:
            last_pool_source["source"] =
"memory"
            last_pool_source["time"] = now_tr()
            last_pool_source["count"] =
len(last_successful_pool)
            return last_successful_pool

        last_pool_source["source"] = "none"
        return []

def get_trading_pool(select_n=30):
    """
        Canlı havuzu üç kademeli öncelikle
        döndürür:
        1. tradingview-scraper (en güvenilir)
        2. stockanalysis.com (yedek kaynak)
        3. son başarılı canlı taramanın
        hafızası (acil durum - SABİT LİSTE DEĞİL)
    """
    raw = get_raw_trading_pool()
    return random.sample(raw, min(select_n,
len(raw))) if raw else []

def get_source_label():
    src = last_pool_source["source"]
    if src == "tradingview":
        return "🐼 Kaynak: Canlı Piyasa
Taraması (TradingView) ✅"
    elif src == "stockanalysis":
        return "🐼 Kaynak: Canlı Piyasa
Taraması (StockAnalysis) ✅"
    elif src == "memory":
        return f"🐼 Kaynak: Son Başarılı

```

```
Tarama Hafızası ⚠️
({last_pool_source['time'].strftime('%H:%M')
) if last_pool_source['time'] else '?'})"
    else:
        return "🐼 Kaynak: Veri alınamadı
❌"
```

```
# =====
# GEÇMİŞ OHLCV VERİSİ – Fallback zinciri:
# 1. stockanalysis.com /history/ sayfası
# (ücretsiz, scraping)
# 2. Twelve Data API (kredi limitli,
# garanti yöntem)
# =====
```

```
def scrape_stockanalysis_history(symbol,
min_rows=150):
    """
```

```
stockanalysis.com/stocks/{symbol}/history/
sayfasından OHLCV tablosunu
    scrape eder. Sayfa varsayılan olarak
~50 satır/sayfa gösteriyor ve
    sayfalama (?p=2, ?p=3...) destekliyor;
yeterli satır birikene kadar
    art arda sayfa çekilir. Sayfa yapısı
değişirse veya erişim engellenirse
    None döner, çağıran taraf Twelve
Data'ya düşer.
```

```
    ÖNEMLİ - PERFORMANS NOTU: Sayfanın
gerçek HTML'i, SvelteKit'in SSR
    yorum işaretleriyle (<!--[!--><!--]-->)
çok yoğun serpiştirilmiş.
```

```
Tek bir büyük regex'te art arda .*? +
re.DOTALL kullanmak burada
CATASTROPHIC BACKTRACKING'e yol açıyor
(test edildi: küçük örnekte
hızlı, gerçek yoğunlukta
dakikalarca/sınırsız sürebiliyor). Bu,
worker
timeout/SIGKILL sorununun GERÇEK kök
nedeniydi. Çözüm: önce TÜM yorum
bloklarını tek seferde (basit, doğrusal
regex ile) temizle, sonra
temiz HTML üzerinde tekrar etmeyen (.*?
zincirsiz) güvenli bir desenle
eşleştir. Bu yöntem mikrosaniyeler
içinde çalışır.
"""
try:
    headers = {"User-Agent":
"Mozilla/5.0 (Windows NT 10.0; Win64;
x64)"}
    all_rows = []
    for page in range(1, 5):
        url =
f"https://stockanalysis.com/stocks/{symbol.
lower()}/history/"
        params = {"p": page} if page >
1 else {}
        r = requests.get(url,
headers=headers, params=params, timeout=8)
        if r.status_code != 200:
            break
        html = r.text

# ADIM 1: SvelteKit SSR yorum
```

```

bloklarını temizle – bu basit desen
    # (. *? ama TEK seferde,
zincirsiz) backtracking riski taşımaz.
    clean_html = re.sub(r'<!--.*?--
>', '', html, flags=re.DOTALL)

    # ADIM 2: Temiz HTML'de art
arda gelen <td> hücrelerini güvenle
eşleştir.

    # Artık aralarda yorum olmadığı
için . *? zincirine gerek yok.
    row_pattern = re.compile(
        r'<td class="sym\[^\"]*">([A-
Za-z]+ \d{1,2}, \d{4})</td>'
        r'<td[^\>]*>([\d.]+)</td>'
        r'<td[^\>]*>([\d.]+)</td>'
        r'<td[^\>]*>([\d.]+)</td>'
        r'<td[^\>]*>([\d.]+)</td>' #
Adj. Close (atlanır)
        r'<td[^\>]*><span[^\>]*>.-?
[\d.]+%</span></td>' # Change (atlanır)
        r'<td[^\>]*>([\d,]+)</td>'
# Volume
    )
    matches =
row_pattern.findall(clean_html)
    if not matches:
        break
    all_rows.extend(matches)
    if len(all_rows) >= min_rows:
        break

    # Bir sonraki sayfada aynı
satırları tekrar görmemek için kısa bekleme

```

```
        if len(all_rows) < 20:
            print(f"[DEBUG
scrape_stockanalysis_history] {symbol}:
sadece {len(all_rows)} satır bulundu
(gerekli: 20+)")
            return None

        df = pd.DataFrame(all_rows,
columns=["Date", "Open", "High", "Low",
"Close", "Volume"])
        df["Date"] =
pd.to_datetime(df["Date"], format="%b %d,
%Y", errors="coerce")
        for col in ["Open", "High", "Low",
"Close"]:
            df[col] =
pd.to_numeric(df[col], errors="coerce")
            df["Volume"] =
df["Volume"].str.replace(",", "",
regex=False)
            df["Volume"] =
pd.to_numeric(df["Volume"],
errors="coerce")
            df = df.dropna(subset=["Date",
"Close"]).drop_duplicates(subset=
["Date"]).sort_values("Date").reset_index(d
rop=True)

        if len(df) < 20:
            print(f"[DEBUG
scrape_stockanalysis_history] {symbol}:
temizlik sonrası {len(df)} satır kaldı
(gerekli: 20+)")
```

```

        return None
        print(f"[DEBUG
scrape_stockanalysis_history] {symbol}:
BAŞARILI, {len(df)} satır")
        return df
    except Exception as e:
        print(f"[DEBUG
scrape_stockanalysis_history] {symbol}:
EXCEPTION {e}")
        return None

# OHLCV verisinin GERÇEK kaynağını takip
eder (pool/havuz kaynağından AYRI).
# get_source_label() pool için,
get_ohlc_source_label() tekil hisse verisi
içindir.
last_ohlc_source = {"source": "none"}

def get_ohlc_source_label():
    src = last_ohlc_source["source"]
    if src == "stockanalysis":
        return "🐼 Kaynak: Canlı Piyasa
Taraması (StockAnalysis) ✅"
    elif src == "twelvedata":
        return "🐼 Kaynak: Twelve Data API
✅"
    else:
        return "🐼 Kaynak: Veri alınamadı
❌"

def td_get_ohlc(symbol, outputsize=210):
    # 1. Önce ücretsiz kaynak:
    stockanalysis.com /history/
    df =

```



```

scrape_stockanalysis_history(symbol,
min_rows=min(outputsize, 150))
    if df is not None and len(df) >= 20:
        last_ohlc_source["source"] =
"stockanalysis"
        print(f"[DEBUG td_get_ohlc]
{symbol}: stockanalysis kullanıldı,
{len(df)} satır")
        return df

    print(f"[DEBUG td_get_ohlc] {symbol}:
stockanalysis başarısız, Twelve Data
deneniyor")
    # 2. Fallback: Twelve Data (kredi
limitli)
    try:
        url =
"https://api.twelvedata.com/time_series"
        params = {
            "symbol": symbol, "interval":
"1day", "outputsize": outputsize,
            "apikey": TD_API_KEY, "format":
"JSON"
        }
        r = requests.get(url,
params=params, timeout=15)
        data = r.json()
        if data.get("status") == "error" or
"values" not in data:
            last_ohlc_source["source"] =
"none"

            print(f"[DEBUG td_get_ohlc]
{symbol}: Twelve Data hatası:
{data.get('message', data)}")

```

```

        return None

        df = pd.DataFrame(data["values"])
        df = df.rename(columns=
{"datetime": "Date", "open": "Open", "high": "Hi
gh", "low": "Low", "close": "Close", "volume": "V
olume"})
        df["Date"] =
pd.to_datetime(df["Date"])
        df =
df.sort_values("Date").reset_index(drop=True)

        for col in
["Open", "High", "Low", "Close", "Volume"]:
            df[col] =
pd.to_numeric(df[col], errors="coerce")
            last_ohlc_source["source"] =
"twelvedata"
            print(f"[DEBUG td_get_ohlc]
{symbol}: Twelve Data kullanıldı, {len(df)}
satır")
            return df
        except:
            last_ohlc_source["source"] =
"none"
            return None

# =====
# GÖSTERGELER
# =====

def calc_rsi(series, period=14):
    delta = series.diff()
    gain =
delta.clip(lower=0).rolling(period).mean()

```

```

        loss = (-
delta.clip(upper=0)).rolling(period).mean()
        rs = gain / loss
        return 100 - (100 / (1 + rs))

def calc_macd(series, fast=12, slow=26,
signal=9):
    ema_fast = series.ewm(span=fast,
adjust=False).mean()
    ema_slow = series.ewm(span=slow,
adjust=False).mean()
    macd = ema_fast - ema_slow
    signal_line = macd.ewm(span=signal,
adjust=False).mean()
    return macd, signal_line

def calc_sma(series, period):
    return series.rolling(period).mean()

def calc_ema(series, period):
    return series.ewm(span=period,
adjust=False).mean()

def calc_atr(df, period=14):
    high, low, close = df['High'],
df['Low'], df['Close']
    prev_close = close.shift(1)
    tr = pd.concat([high-low, (high-
prev_close).abs(), (low-prev_close).abs()],
axis=1).max(axis=1)
    return tr.rolling(period).mean()

def calc_adx(df, period=14):
    high, low, close = df['High'],

```

```

df['Low'], df['Close']
    plus_dm = high.diff()
    minus_dm = -low.diff()
    plus_dm[plus_dm < 0] = 0
    minus_dm[minus_dm < 0] = 0
    plus_dm[(plus_dm - minus_dm) < 0] = 0
    minus_dm[(minus_dm - plus_dm) < 0] = 0
    prev_close = close.shift(1)
    tr = pd.concat([high-low, (high-
prev_close).abs(), (low-prev_close).abs()],
axis=1).max(axis=1)
    atr = tr.rolling(period).mean()
    plus_di = 100 *
(plus_dm.rolling(period).mean() / atr)
    minus_di = 100 *
(minus_dm.rolling(period).mean() / atr)
    dx = 100 * (plus_di - minus_di).abs() /
(plus_di + minus_di)
    return dx.rolling(period).mean()

def detect_candle_pattern(df):
    """Basit mum formasyonu tespiti -
hammer ve engulfing"""
    try:
        last = df.iloc[-1]
        prev = df.iloc[-2]
        body = abs(last['Close'] -
last['Open'])
        full_range = last['High'] -
last['Low']
        lower_wick = min(last['Open'],
last['Close']) - last['Low']

        # Hammer: küçük gövde, uzun alt

```

```

fital
    is_hammer = full_range > 0 and
lower_wick > body * 2 and body < full_range
* 0.35

    # Bullish engulfing: önceki kırmızı
mum, şimdiki yeşil mum onu tamamen kapsıyor
    prev_bearish = prev['Close'] <
prev['Open']
    curr_bullish = last['Close'] >
last['Open']
    is_engulfing = prev_bearish and
curr_bullish and last['Close'] >
prev['Open'] and last['Open'] <
prev['Close']

    if is_hammer:
        return "🔨 Hammer (Dönüş
formasyonu)"
    elif is_engulfing:
        return "📈 Bullish Engulfing
(Güçlü dönüş)"
    return None
except:
    return None

def get_weekly_rsi(symbol, df=None):
    """Haftalık RSI - çoklu zaman dilimi
teyidi için"""
    try:
        if df is None:
            df = td_get_ohlcv(symbol,
outputsize=100)
        if df is None or len(df) < 30:

```

```

        return None
    weekly =
df.set_index('Date').resample('W').agg({

'Open':'first','High':'max','Low':'min','Cl
ose':'last','Volume':'sum'
}).dropna()
    if len(weekly) < 15:
        return None
    rsi = calc_rsi(weekly['Close'])
    return float(rsi.iloc[-1])
except:
    return None

def get_qqq_performance(days=20):
    try:
        df = td_get_ohlcv('QQQ',
outputsize=days+5)
        if df is None or len(df) < days:
            return None
        old_price =
float(df['Close'].iloc[-days])
        new_price =
float(df['Close'].iloc[-1])
        return (new_price - old_price) /
old_price * 100
    except:
        return None

# =====
# ELLIOTT DALGA SAYIMI
# =====
# Not: Tam otomatik Elliott dalga
etiketleme algoritmik olarak çok zor bir

```

```
problemdir.
# Bu modül, ZigZag pivot tespiti üzerine
basitleştirilmiş bir 5-dalga kontrolü kurar
# ve sonucu bir ÖNERİ olarak sunar;
kesin/garanti bir dalga sayımı değildir.

def find_zigzag_pivots(df,
pct_threshold=5.0):
    """
        Basit bir ZigZag pivot tespiti: fiyat
        önceki pivot noktasından
        en az pct_threshold% hareket etmedikçe
        yeni bir pivot kabul edilmez.
        Küçük gürültüyü eler, anlamlı yön
        değişimlerini (swing high/low) yakalar.
    """
    closes = df['Close'].values
    dates = df['Date'].values
    if len(closes) < 10:
        return []

    pivots = []
    last_pivot_idx = 0
    last_pivot_price = closes[0]
    direction = None # 'up' veya 'down'

    for i in range(1, len(closes)):
        change_pct = (closes[i] -
last_pivot_price) / last_pivot_price * 100

        if direction is None:
            if abs(change_pct) >=
pct_threshold:
                direction = 'up' if
```

```

change_pct > 0 else 'down'
        last_pivot_idx = i
        last_pivot_price =
closes[i]
        elif direction == 'up':
            if closes[i] >
last_pivot_price:
                last_pivot_idx = i
                last_pivot_price =
closes[i]
            elif (closes[i] -
last_pivot_price) / last_pivot_price * 100
<= -pct_threshold:
                pivots.append({'idx':
last_pivot_idx, 'price': last_pivot_price,
'type': 'high', 'date':
dates[last_pivot_idx]})
                direction = 'down'
                last_pivot_idx = i
                last_pivot_price =
closes[i]
            elif direction == 'down':
                if closes[i] <
last_pivot_price:
                    last_pivot_idx = i
                    last_pivot_price =
closes[i]
                elif (closes[i] -
last_pivot_price) / last_pivot_price * 100
>= pct_threshold:
                    pivots.append({'idx':
last_pivot_idx, 'price': last_pivot_price,
'type': 'low', 'date':
dates[last_pivot_idx]})

```



```

        direction = 'up'
        last_pivot_idx = i
        last_pivot_price =
closes[i]

    # Son (henüz tamamlanmamış) pivotu da
    ekle
    pivots.append({'idx': last_pivot_idx,
'price': last_pivot_price,
                    'type': 'high' if
direction == 'up' else 'low', 'date':
dates[last_pivot_idx]})
    return pivots

def label_elliott_waves(pivots):
    """
        Son 6 pivot noktasını (5 dalga +
başlangıç) kullanarak klasik dürtü
(impulse)
        yapısının temel kurallarını kontrol
eder:
        - Dalga 2, Dalga 1'in başlangıcının
altına (boğa) / üstüne (ayı) inmez
        - Dalga 3, Dalga 1'den daha uzundur
(genelde en uzun dalga)
        - Dalga 4, Dalga 1'in zirvesiyle
çakışmaz (overlap kuralı)
        Kurallara uyan en güncel 6 pivot
bulunursa "5 dalga tamamlandı" önerisi
yapılır.
    """
    if len(pivots) < 6:
        return None

```

```

last6 = pivots[-6:]
p0, p1, p2, p3, p4, p5 = [p['price']
for p in last6]
    bullish = p1 > p0 # ilk hareket
yukarıysa boğa dürtüsü olabilir

    if bullish:
        wave1 = p1 - p0
        wave3 = p3 - p2
        wave5 = p5 - p4
        rule_wave2 = p2 > p0 #
Dalga 2, başlangıcın altına inmemeli
        rule_wave3_longest = wave3 > wave1
and wave3 > wave5
        rule_wave4_no_overlap = p4 > p1 #
Dalga 4, Dalga 1 zirvesiyle çakışmamalı
        valid = rule_wave2 and
rule_wave3_longest and
rule_wave4_no_overlap and p5 > p3
        direction_text = "📈 YUKARI (🐂
Boğa Dürtüsü)"
        next_expectation = "5 dalga
tamamlanmış görünüyor – ABC düzeltmesi
(aşağı) beklenebilir"
    else:
        wave1 = p0 - p1
        wave3 = p2 - p3
        wave5 = p4 - p5
        rule_wave2 = p2 < p0
        rule_wave3_longest = wave3 > wave1
and wave3 > wave5
        rule_wave4_no_overlap = p4 < p1
        valid = rule_wave2 and
rule_wave3_longest and

```

```

rule_wave4_no_overlap and p5 < p3
    direction_text = "📉 AŞAĞI (🐻 Ayı
Dürtüsü)"
    next_expectation = "5 dalga
tamamlanmış görünüyor – ABC düzeltmesi
(yukarı) beklenebilir"

    return {
        'valid': valid,
        'bullish': bullish,
        'direction_text': direction_text,
        'next_expectation':
next_expectation,
        'pivots': last6,
        'rule_wave2': rule_wave2,
        'rule_wave3_longest':
rule_wave3_longest,
        'rule_wave4_no_overlap':
rule_wave4_no_overlap,
    }

def elliott_wave_analysis(symbol):
    """
    /elliott (ve eş değerleri: /eliot
/elliott /dalga) komutu için ana fonksiyon.
    Hem manuel tekil sorguda hem otomatik
taramada kullanılır.
    Döner: {"signal": <5 dalga tamamlanmış
öneri metni veya None>, "info": <her
durumda gösterilebilecek özet>}
    """
    try:
        df = td_get_ohlc(symbol,
outputsize=150)

```

```
if df is None:
    return None
if len(df) < 30:
    return {"signal": None, "info":
None, "ipo": True, "ipo_message":
ipo_short_data_message(symbol, len(df),
30)}
```

```
price = float(df['Close'].iloc[-1])
pivots = find_zigzag_pivots(df,
pct_threshold=5.0)
```

```
if len(pivots) < 6:
    info = f"""
```

 **ELLIOTT DALGA ANALİZİ – Bilgi**

Amaçlı

 **{symbol}**

 Fiyat: \${price:.2f}

 Tespit edilen pivot sayısı:
{len(pivots)} (gerekli: 6+)

 **Özet:** Anlamlı bir dalga sayımı için yeterli sayıda belirgin yön değişimi (pivot) bulunamadı. Bu genelde fiyatın yatay/düşük volatiliteli bir bantta hareket ettiği anlamına gelir.

 Bu otomatik bir öneridir, kesin dalga sayımı değildir.

 {now_tr().strftime('%d.%m.%Y %H:%M')}
"""

```
return {"signal": None, "info":
```

```

info}

        result =
label_elliott_waves(pivots)
        if result is None:
            return None

        pivot_lines = "\n".join(
            [f"    {i}. {p['date']} -
${p['price']:.2f} ({'Tepe' if
p['type']=='high' else 'Dip'})"
            for i, p in
enumerate(result['pivots'])]
        )

        kural_text = (
            f"🌊 Dalga 2 kuralı: {'✅' if
result['rule_wave2'] else '❌'}\n"
            f"🌊 Dalga 3 en uzun: {'✅' if
result['rule_wave3_longest'] else '❌'}\n"
            f"🌊 Dalga 4 çakışma yok: {'✅'
if result['rule_wave4_no_overlap'] else
'❌'}"
        )

        if result['valid']:
            msg = f"""
🌊 <b>ELLIOTT DALGA SAYIMI – 5 Dalga
Tamamlanma Önerisi</b>
🚩 <b>{symbol} - {result['direction_text']}
</b>

$ Güncel Fiyat: ${price:.2f}

```

📌 **Tespit Edilen Pivot Noktaları (0-5):**

{pivot_lines}

✅ **Dürtü Dalgası Kuralları:
{kural_text}**

🎯 **Beklenti:
{result['next_expectation']}**

⚠️ Bu otomatik bir öneridir, kesin dalga sayımı garantisi vermez.
Lütfen kendi teyidini de yap.
🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}
"""

return {"signal": msg, "info":
None}
else:
info = f"""

🌐 **ELLIOTT DALGA ANALİZİ – Bilgi Amaçlı**
**(Kurallar Tam Karşılanmadı)
🦅 **{symbol}****

💰 Fiyat: \${price:.2f}

📌 **Tespit Edilen Pivot Noktaları (0-5):**

{pivot_lines}

📋 **Dürtü Dalgası Kuralları:
{kural_text}**

💬 **Özet:** Son 6 pivot, klasik 5 dalga dürtü yapısının kurallarını

tam karşılamıyor. Bu, ya dalga sayımının farklı bir noktadan başlaması gerektiği ya da şu an düzeltme (ABC) aşamasında olduğu anlamına gelebilir.

⚠ Bu otomatik bir öneridir, kesin dalga sayımı değildir.

🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}\n""

```
        return {"signal": None, "info": info}\n    except:\n        return None
```

=====

ELLİOTT OTOMATİK TARAMA – Losers + 4
Destekleyici Kriter

=====

Bu, manuel /dalga komutundan AYRI bir mimari. Rastgele hisse seçmek
yerine, sistematik olarak "en çok kaybeden" (losers) listesinden
tarar – mantık: yukarı dönüş potansiyeli en çok düşmüş hisselerde
aranır, rastgele bir hissede değil. 4 destekleyici kriterden en az 3'ü
karşılanırsa + ZigZag pivot dönüşü tespit edilirse bildirim gönderilir.
Dalga numarası (1,3,5) iddia edilmez – bu, ihtiyatlı/dürüst bir
"potansiyel dönüş" önerisidir, kesin Elliott sayımı değildir.

```
def check_reversal_criteria(df,
symbol=None):
    """
    4 destekleyici kriteri kontrol eder ve
    kaçının karşılandığını döner.
    Döner: (karşılanan_sayı, detaylar_dict)
    """
    try:
        close = df['Close']
        volume = df['Volume']
        price = float(close.iloc[-1])

        # Kriter 1: RSI aşırı satım
        rsi = calc_rsi(close)
        rsi_now = float(rsi.iloc[-1])
        crit_rsi = rsi_now < 35

        # Kriter 2: Hacim azalması (son 3
        günün hacmi, önceki 3 güne göre düşüyor mu)
        # Kapitulasyon sonrası "satış
        baskısının tükenmesi" işareti.
        vol_recent =
        volume.iloc[-3:].mean()
        vol_prior =
        volume.iloc[-6:-3].mean() if len(volume) >=
        6 else vol_recent
        crit_vol_decline = vol_recent <
        vol_prior if vol_prior > 0 else False

        # Kriter 3: Ardışık düşüş günleri
        (son 8 günün en az %60'ı düşüş)
        last_8 = close.iloc[-9:].values if
        len(close) >= 9 else close.values
        down_days = sum(1 for i in range(1,
```



```

len(last_8)) if last_8[i] < last_8[i-1])
    total_days = len(last_8) - 1
    crit_down_streak = total_days > 0
and (down_days / total_days) >= 0.6

    # Kriter 4: Destek seviyesine
yakınlık (SMA200)
    sma200 = calc_sma(close, min(200,
len(close)-1))
    if len(close) >= 190:
        sma200_now =
float(sma200.iloc[-1]) if not
pd.isna(sma200.iloc[-1]) else price
    else:
        barchart_sma200 =
get_sma200_barchart(symbol) if symbol else
None
        sma200_now = barchart_sma200 if
barchart_sma200 is not None else (
            float(sma200.iloc[-1]) if
not pd.isna(sma200.iloc[-1]) else price
        )
        dist_sma200 = abs(price -
sma200_now) / price * 100
        crit_near_support = dist_sma200 < 5

    confirmed = sum([crit_rsi,
crit_vol_decline, crit_down_streak,
crit_near_support])
    details = {
        "rsi_now": rsi_now, "crit_rsi":
crit_rsi,
        "vol_recent": vol_recent,
"vol_prior": vol_prior, "crit_vol_decline":

```

```

crit_vol_decline,
        "down_days": down_days,
"total_days": total_days,
"crit_down_streak": crit_down_streak,
        "dist_sma200": dist_sma200,
"crit_near_support": crit_near_support,
    }
    return confirmed, details
except:
    return 0, {}

def elliott_auto_scan_candidate(symbol):
    """
    Tek bir sembol için: önce 4
    destekleyici kriteri kontrol eder (en az 3
    gerekli), sonra ZigZag pivot ile gerçek
    bir yukarı dönüş başlayıp
    başlamadığına bakar. İkisi de tutarsa
    bildirim mesajı döner.
    """
    try:
        df = td_get_ohlc(symbol,
        outputsize=60)
        if df is None or len(df) < 20:
            return None

        confirmed, details =
        check_reversal_criteria(df, symbol=symbol)
        if confirmed < 3:
            return None

        pivots = find_zigzag_pivots(df,
        pct_threshold=4.0)
        if len(pivots) < 2:

```

```
return None

# Son pivot bir dip (low) olmalı VE
fiyat o dipten beri yükseliyor olmalı
last_pivot = pivots[-1]
if last_pivot['type'] != 'low':
    return None
price_now =
float(df['Close'].iloc[-1])
if price_now <=
last_pivot['price']:
    return None # henüz gerçek bir
yön değişimi yok
```

```
price = price_now
```

```
msg = f"""
```

 POTANSİYEL YUKARI DÖNÜŞ – ZigZag
Pivot

 {symbol}

 Fiyat: \${price:.2f}

 Destekleyici Kriterler ({confirmed}/4):

 RSI Aşırı Satım: {""} if
details['crit_rsi'] else ""

{details['rsi_now']:.1f} / Ref: 35 altı

 Hacim Azalması (Kapitülasyon): {""} if
details['crit_vol_decline'] else ""} Son
3 gün ort. {details['vol_recent']:.0f} /
Önceki 3 gün ort.

{details['vol_prior']:.0f}

 Ardışık Düşüş Günleri: {""} if
details['crit_down_streak'] else ""
{details['down_days']}/{details['total_days']}

```

    ']} gün düşüş
    🛡 Destek Seviyesine Yakınlık (SMA200):
    {"✅" if details['crit_near_support'] else
    "❌"} %{details['dist_sma200']:.1f}
    uzaklıkta / Ref: %5 altı

    🌀 ZigZag Pivot: Yeni dip oluştu
    (${last_pivot['price']:.2f}), fiyat şu an
    üstünde

    ⚠ Bu otomatik bir öneridir, kesin dalga
    sayımı veya garanti dönüş teyidi değildir.
    🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}
    """

    return {"signal": msg, "info":
None}
    except:
        return None

def elliott_auto_scan_loop():
    """
    Manuel /dalga komutundan TAMAMEN AYRI
    çalışan döngü. Her 2.5 saatte
    bir (piyasa açıkken), losers listesinin
    ilk 50-100 sembolünü tarar.
    Bu, ana auto_scan_loop'tan (25
    dakikalık) bağımsız bir thread'dir,
    böylece kredi/worker yükü ayrı
    yönetilir.
    """
    time.sleep(60) # ana döngüyle
    çakışmasın, biraz geriden başlasın
    while True:
        try:

```

```

        if is_bot_active():
            losers =
fetch_losers_ranked(limit=100)
            if losers:
                found = 0
                for ticker in losers:
                    result =
elliott_auto_scan_candidate(ticker)
                    if result and
result.get("signal"):

send_kanal(result["signal"], "elliott")

send_news_for_signal(ticker, "Elliott
Otomatik Tarama")

                    found += 1
                    time.sleep(2)
                    time.sleep(0.4)
                    print(f"[DEBUG
elliott_auto_scan_loop] Tarama tamamlandı,
{found} bildirim gönderildi, {len(losers)}
hisse tarandı")
            else:
                print("[DEBUG
elliott_auto_scan_loop] Losers listesi
alınamadı, bu döngü atlandı")
            except Exception as e:
                print(f"[DEBUG
elliott_auto_scan_loop] EXCEPTION: {e}")
                time.sleep(2.5 * 60 * 60) # 2.5
saat

# =====
# FORMASYON SİNYALİ (Dönüş + Devam

```

```
Formasyonları)
# =====
# Bu kategori, klasik grafik
formasyonlarını tespit eder ve her
formasyon
# için beklenen yönü (yukarı/aşağı/devam)
belirtir. TREND SİNYALİ ve ERKEN
# UYARI'dan tamamen ayrı, kendi başına
çalışan üçüncü bir sinyal türüdür.
#
# DÖNÜŞ FORMASYONLARI: Wolfe Wave,
Rising/Falling Wedge, Head&Shoulders
# (+ ters), Diamond
Top/Bottom, Three Drives
# DEVAM FORMASYONLARI: Triangle
(simetrik), Flag, Rectangle

FORMASYON_YON_BILGISI = {
    'wolfe_wave': {
        'isim': 'Wolfe Wave',
        'kategori': 'Dönüş',
    },
    'rising_wedge': {
        'isim': 'Rising Wedge (Yükselen
Takoz)',
        'kategori': 'Dönüş',
        'yon': 'AŞAĞI 📉',
        'not': 'Yükselen takozlar genelde
aşağı kırılımla sonuçlanır',
    },
    'falling_wedge': {
        'isim': 'Falling Wedge (Düşen
Takoz)',
        'kategori': 'Dönüş',
```

```
        'yon': 'YUKARI ⬆️',
        'not': 'Düşen takozlar genelde
yukarı kırılımla sonuçlanır',
    },
    'head_shoulders': {
        'isim': 'Head & Shoulders (Omuz Baş
Omuz)',
        'kategori': 'Dönüş',
        'yon': 'AŞAĞI ⬇️',
        'not': 'Yükseliş trendinin sonunda
görülen klasik dönüş formasyonu',
    },
    'inverse_head_shoulders': {
        'isim': 'Ters Omuz Baş Omuz',
        'kategori': 'Dönüş',
        'yon': 'YUKARI ⬆️',
        'not': 'Düşüş trendinin sonunda
görülen klasik dönüş formasyonu',
    },
    'diamond_top': {
        'isim': 'Diamond Top',
        'kategori': 'Dönüş',
        'yon': 'AŞAĞI ⬇️',
        'not': 'Yükseliş trendinin
tepesinde genişleyip daralan bir yapı',
    },
    'diamond_bottom': {
        'isim': 'Diamond Bottom',
        'kategori': 'Dönüş',
        'yon': 'YUKARI ⬆️',
        'not': 'Düşüş trendinin dibinde
genişleyip daralan bir yapı',
    },
    'three_drives_bear': {
```

```
      'isim': 'Three Drives (Tükenme -  
Ayı)',  
      'kategori': 'Dönüş',  
      'yon': 'AŞAĞI ⬇️',  
      'not': '3 itme dalgalı yukarı  
tükenme, genelde sert düşüşle sonuçlanır',  
    },  
    'three_drives_bull': {  
      'isim': 'Three Drives (Tükenme -  
Boğa)',  
      'kategori': 'Dönüş',  
      'yon': 'YUKARI ⬆️',  
      'not': '3 itme dalgalı aşağı  
tükenme, genelde sert yükselişle  
sonuçlanır',  
    },  
    'triangle': {  
      'isim': 'Symmetrical Triangle  
(Simetrik Üçgen)',  
      'kategori': 'Devam',  
      'yon': 'MEVCUT TREND YÖNÜNDE ➡️',  
      'not': 'Daralan üçgen genelde  
mevcut trendin devamı ile kırılır',  
    },  
    'flag': {  
      'isim': 'Flag / Pennant  
(Bayrak/Flama)',  
      'kategori': 'Devam',  
      'yon': 'MEVCUT TREND YÖNÜNDE ➡️',  
      'not': 'Kısa bir konsolidasyon  
sonrası trend genelde devam eder',  
    },  
    'rectangle': {  
      'isim': 'Rectangle (Dikdörtgen)',
```



```

        'kategori': 'Devam',
        'yon': 'MEVCUT TREND YÖNÜNDE ➡',
        'not': 'Yatay bant sonrası fiyat
genelde geldiği yöne devam eder',
    },
}

def _pct_diff(a, b):
    return abs(a - b) / ((a + b) / 2) * 100
if (a + b) != 0 else 999

def detect_formation(pivots,
trend_direction=None):
    """
        Son pivot noktalarını geometrik olarak
        inceleyip bilinen formasyonlara
        uyup uymadığını kontrol eder. Birden
        fazla formasyon eşleşebilir,
        hepsi liste olarak döner (boş liste =
        formasyon tespit edilmedi).
        trend_direction: 'up' / 'down' / None –
        devam formasyonları için mevcut
        trendin yönünü belirtir (örn. Üçgen
        kırılımının hangi yöne olası olduğu).
    """
    found = []
    if len(pivots) < 5:
        return found

    last5 = pivots[-5:]
    prices = [p['price'] for p in last5]
    p0, p1, p2, p3, p4 = prices

    # --- DÖNÜŞ: Head & Shoulders (Tepe-

```

```
Omuz-Baş-Omuz-Tepe: yüksek-düşük-yüksek-  
düşük-yüksek) ---  
    types = [p['type'] for p in last5]  
    if types ==  
['low','high','low','high','low']:  
        # p1 (sol omuz), p3 (sağ omuz)  
        tepeleri; ortadaki p2 dip, ama "baş" aranan  
        asıl tepe ayrı pivot setinde olur  
        pass  
  
    if len(pivots) >= 5:  
        # Tepe noktalarını (high) ve dip  
        noktalarını (low) ayır, son 3 tepe ile H&S  
        dene  
        highs = [p for p in pivots[-7:] if  
p['type'] == 'high']  
        lows = [p for p in pivots[-7:] if  
p['type'] == 'low']  
  
        if len(highs) >= 3:  
            h1, h2, h3 = highs[-3]  
['price'], highs[-2]['price'], highs[-1]  
['price']  
            # Baş (h2) belirgin şekilde  
            diğer ikisinden yüksek, omuzlar (h1, h3)  
            birbirine yakın  
            if h2 > h1 * 1.03 and h2 > h3 *  
1.03 and _pct_diff(h1, h3) < 4:  
  
            found.append('head_shoulders')  
  
        if len(lows) >= 3:  
            l1, l2, l3 = lows[-3]['price'],  
lows[-2]['price'], lows[-1]['price']
```

```

        if l2 < l1 * 0.97 and l2 < l3 *
0.97 and _pct_diff(l1, l3) < 4:

found.append('inverse_head_shoulders')

        # --- Wedge (Takoz): art arda gelen
tepeler VE dipler aynı yönde daralarak
hareket ediyor ---
        if len(highs) >= 2 and len(lows) >=
2:
            highs_rising = highs[-1]
['price'] > highs[-2]['price']
            lows_rising = lows[-1]['price']
> lows[-2]['price']
            # Rising Wedge: hem tepeler hem
dippler yükseliyor ama aralık daralıyor
(momentum azalıyor)
            if highs_rising and
lows_rising:
                range_old = highs[-2]
['price'] - lows[-2]['price']
                range_new = highs[-1]
['price'] - lows[-1]['price']
                if range_new < range_old *
0.85:

found.append('rising_wedge')
            # Falling Wedge: hem tepeler
hem dipler düşüyor ama aralık daralıyor
            elif not highs_rising and not
lows_rising:
                range_old = highs[-2]
['price'] - lows[-2]['price']
                range_new = highs[-1]

```

```

['price'] - lows[-1]['price']
        if range_new < range_old *
0.85:

found.append('falling_wedge')

        # --- Diamond Top/Bottom: aralık
önce genişler sonra daralır ---
        if len(highs) >= 2 and len(lows) >=
2:
            range1 = highs[-2]['price'] -
lows[-2]['price']
            range2 = highs[-1]['price'] -
lows[-1]['price']
            # Genişleyip daralma paterni
(basitleştirilmiş): son aralık ortadaki
genişlemeden belirgin küçük
            if len(highs) >= 3 and
len(lows) >= 3:
                range0 = highs[-3]['price']
- lows[-3]['price']
                if range1 > range0 * 1.2
and range2 < range1 * 0.7:
                    if highs[-1]['idx'] >
lows[-1]['idx']:

found.append('diamond_top' if
trend_direction == 'up' else
'diamond_bottom')

        # --- Three Drives: 3 ardışık tepe
(veya dip) her biri öncekinden daha
yüksek/düşük ---
        if len(highs) >= 3:

```

```

        h1, h2, h3 = highs[-3]
        ['price'], highs[-2]['price'], highs[-1]
        ['price']

        if h1 < h2 < h3:

found.append('three_drives_bear')
        if len(lows) >= 3:
            l1, l2, l3 = lows[-3]['price'],
lows[-2]['price'], lows[-1]['price']
            if l1 > l2 > l3:

found.append('three_drives_bull')

        # --- Triangle (devam): tepeler
        düşüyor, dipler yükseliyor (daralan yatay
        bant) ---
        if len(highs) >= 2 and len(lows) >=
2:
            if highs[-1]['price'] <
highs[-2]['price'] and lows[-1]['price'] >
lows[-2]['price']:
                found.append('triangle')

        # --- Rectangle (devam): son
        tepeler birbirine yakın VE son dipler
        birbirine yakın ---
        if len(highs) >= 2 and len(lows) >=
2:
            if _pct_diff(highs[-1]
['price'], highs[-2]['price']) < 2 and
_pct_diff(lows[-1]['price'], lows[-2]
['price']) < 2:
                found.append('rectangle')

```

```

        return list(set(found))

def formation_analysis(symbol):
    """
        /formasyon (ve eş değerleri: /formation
        /pattern) komutu için ana fonksiyon.
        Hem manuel tekil sorguda hem otomatik
        taramada kullanılır.
    """
    try:
        df = td_get_ohlc(symbol,
        outputsize=150)
        if df is None:
            return None
        if len(df) < 30:
            return {"signal": None, "info":
None, "ipo": True, "ipo_message":
ipo_short_data_message(symbol, len(df),
30)}

        price = float(df['Close'].iloc[-1])
        close = df['Close']
        sma50 = calc_sma(close, min(50,
len(close)-1))
        sma50_now = float(sma50.iloc[-1])
        if not pd.isna(sma50.iloc[-1]) else price
            trend_direction = 'up' if price >
sma50_now else 'down'

        pivots = find_zigzag_pivots(df,
pct_threshold=4.0)
        formations =
detect_formations(pivots, trend_direction)

```

```

        if not formations:
            info = f"""
 <b>FORMASYON ANALİZİ – Bilgi Amaçlı</b>
 <b>{symbol}</b>



---



$ Fiyat: ${price:.2f}
 Genel Trend: {"Yukarı" if
trend_direction=='up' else "Aşağı"}
(SMA50'ye göre)

💬 <b>Özet:</b> Şu an bilinen dönüş veya
devam formasyonlarından
(Wolfe Wave, Wedge, H&S, Diamond, Three
Drives, Triangle, Flag,
Rectangle) hiçbirisi net şekilde tespit
edilemedi.

⚠ Bu otomatik bir öneridir, kesin
formasyon teyidi değildir.
🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}
"""

        return {"signal": None, "info":
info}

    bloklar = []
    for key in formations:
        bilgi =
FORMASYON_YON_BILGISI.get(key)
        if not bilgi:
            continue
        yon = bilgi.get('yon',
'Belirsiz')
        not_text = bilgi.get('not', '')
        bloklar.append(

```

```

        f"🔍 <b>{bilgi['isim']}</b>
</b>\n"

        f" 📊 Kategori:
{bilgi['kategori']} Formasyonu\n"
        f" 🎯 Beklenen Yön:
{yon}\n"

        f" 📄 {not_text}"
    )

    formasyon_text =
"\n\n".join(bloklar)

    msg = f"""
📌 <b>FORMASYON SİNYALİ</b>
🚨 <b>{symbol}</b> - Formasyon(lar) Tespit
Edildi</b>



---


💰 Fiyat: ${price:.2f}
📊 Genel Trend: {"Yukarı" if
trend_direction=='up' else "Aşağı"}
(SMA50'ye göre)

{formasyon_text}

⚠️ Bu otomatik bir öneridir, kesin
formasyon teyidi değildir.
Lütfen grafiği kendin de görsel olarak
teyit et.
🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}
"""

    return {"signal": msg, "info":
None}
except:
    return None

```



```

# =====
# ANA ANALİZ (Trend Sinyali)
# =====

def ipo_short_data_message(symbol,
data_days, min_days):
    return (
        f"📰 <b>{symbol} – Yeni IPO  

        Olabilir</b>\n"
        f"Sadece {data_days} günlük veri  

        mevcut (gerekli: en az {min_days} gün).\n"
        f"Hisse yakın zamanda halka arz  

        olmuş olabilir, sağlıklı analiz için  

        yeterli geçmiş veri yok."
    )

def analyze_stock(symbol, df=None):
    try:
        if df is None:
            df = td_get_ohlc(symbol,
outputsize=210)
        if df is None:
            print(f"[DEBUG analyze_stock]
{symbol}: df is None (veri çekilemedi)")
            return None

        data_days = len(df)
        if data_days < 20:
            print(f"[DEBUG analyze_stock]
{symbol}: data_days={data_days} < 20,
yetersiz veri (muhtemel yeni IPO)")
            return {"signal": None, "info":

```

```
None, "ipo": True, "ipo_message":  
ipo_short_data_message(symbol, data_days,  
20)}
```

```
        close = df['Close']  
        volume = df['Volume']  
  
        rsi = calc_rsi(close)  
        macd, macd_signal =  
calc_macd(close)  
        ema20 = calc_ema(close, 20)  
        sma50 = calc_sma(close, min(50,  
data_days-1))  
        sma200 = calc_sma(close, min(200,  
data_days-1))  
        vol_ma20 = volume.rolling(min(20,  
data_days-1)).mean()  
        atr = calc_atr(df, period=min(14,  
data_days-1))  
        adx = calc_adx(df, period=min(14,  
data_days-1))  
  
        price = float(close.iloc[-1])  
        rsi_now = float(rsi.iloc[-1])  
        rsi_prev = float(rsi.iloc[-2])  
        macd_now = float(macd.iloc[-1])  
        macd_sig_now =  
float(macd_signal.iloc[-1])  
        macd_prev_val =  
float(macd.iloc[-2])  
        macd_sig_prev =  
float(macd_signal.iloc[-2])  
        ema20_now = float(ema20.iloc[-1])  
        sma50_now = float(sma50.iloc[-1])
```

```

if not pd.isna(sma50.iloc[-1]) else price

        # SMA200: Önce gerçek 200 günlük
veri varsa onu kullan (en doğru).
        # Veri yetersizse (örn.
stockanalysis.com'un ~50 günlük limiti),
        # Barchart.com'un hazır hesapladığı
SMA200 değerine düş.
        # "Sınırlı veri" uyarısı SADECE
Barchart fallback'i de başarısız
        # olup ham veriden hesaplamak
zorunda kaldığımızda gösterilir –
        # çünkü o durumda SMA200 güvenilirmez
olur, diğer göstergeler değil.
        low_data_warning = False
        if data_days >= 190:
            sma200_now =
float(sma200.iloc[-1]) if not
pd.isna(sma200.iloc[-1]) else price
        else:
            barchart_sma200 =
get_sma200_barchart(symbol)
            if barchart_sma200 is not None:
                sma200_now =
barchart_sma200
            else:
                sma200_now =
float(sma200.iloc[-1]) if not
pd.isna(sma200.iloc[-1]) else price
                low_data_warning = True #
SMA200 güvenilir kaynaktan gelmedi, ham
veriden tahmin edildi

        vol_now = float(volume.iloc[-1])

```

```

        vol_avg = float(vol_ma20.iloc[-1])
    if not pd.isna(vol_ma20.iloc[-1]) else
    vol_now

        atr_now = float(atr.iloc[-1]) if
    not pd.isna(atr.iloc[-1]) else price * 0.02
        adx_now = float(adx.iloc[-1]) if
    not pd.isna(adx.iloc[-1]) else 20

    trend_sma200 = f"✅ Fiyat
    ${price:.2f} / Ortalama ${sma200_now:.2f}
    → Üstünde (İyi)" if price > sma200_now
    else f"⚠️ Fiyat ${price:.2f} / Ortalama
    ${sma200_now:.2f} → Altında"
        trend_ema20 = f"✅ Fiyat
    ${price:.2f} / Ortalama ${ema20_now:.2f} →
    Üstünde (İyi)" if price > ema20_now else
    f"⚠️ Fiyat ${price:.2f} / Ortalama
    ${ema20_now:.2f} → Altında"
        trend_sma50 = f"✅ Fiyat
    ${price:.2f} / Ortalama ${sma50_now:.2f} →
    Üstünde (İyi)" if price > sma50_now else
    f"⚠️ Fiyat ${price:.2f} / Ortalama
    ${sma50_now:.2f} → Altında"
        resistance_risk = (
            (abs(price - sma50_now) / price
            < 0.02 and price < sma50_now) or
            (abs(price - sma200_now) /
            price < 0.02 and price < sma200_now)
        )

    # 🛡️ Destek/Direnç dinamik başlık
    ve metin
        sma200_label = "Destek" if price >=
    sma200_now else "Direnç"

```

```

        ema20_label = "Destek" if price >=
ema20_now else "Direnç"
        sma200_dist = abs(price -
sma200_now) / price * 100
        ema20_dist = abs(price - ema20_now)
/ price * 100
        both_same = sma200_label ==
ema20_label
        if both_same and sma200_label ==
"Destek":
            sd_baslik = "🛡️ Destek (Yakın
Destek Noktası)"
            elif both_same and sma200_label ==
"Direnç":
                sd_baslik = "🛡️ Direnç (Yakın
Direnç Noktası)"
            else:
                sd_baslik = "🛡️ Destek/Direnç
Kontrolü"
        near = sma200_dist < 3 or
ema20_dist < 2
        sd_text = (
            f"{'✅' if near else '❌'}
SMA200 [{sma200_label}] %{sma200_dist:.1f}
uzaklıkta / "
            f"EMA20 [{ema20_label}] %
{ema20_dist:.1f} uzaklıkta → {'Yakın
(İyi)' if near else 'Uzak'}"
        )

        rsi_signal = rsi_now > rsi_prev and
40 < rsi_now < 65
        macd_crossover = macd_prev_val <
macd_sig_prev and macd_now > macd_sig_now

```

```

        macd_positive = macd_now >
macd_sig_now
        vol_ratio = vol_now / vol_avg if
vol_avg > 0 else 0
        vol_ok = vol_ratio >= 1.5

        # 💰 Dolar bazlı işlem hacmi (Adet
× Fiyat) – SADECE BİLGİ AMAÇLI, skora
katkısı YOK.
        # Mutlak bir referans (örn.
$100.000) yerine ÖNCEKİ GÜNE göre
karşılaştırma gösterilir.
        dollar_volume = vol_now * price
        if dollar_volume >= 1_000_000_000:
            dollar_vol_str =
f"${dollar_volume/1_000_000_000:.2f}
Milyar"
        elif dollar_volume >= 1_000_000:
            dollar_vol_str =
f"${dollar_volume/1_000_000:.2f} Milyon"
        else:
            dollar_vol_str =
f"${dollar_volume:,.0f}"

        prev_dollar_change_text = ""
        try:
            if len(df) >= 2:
                vol_prev_day =
float(volume.iloc[-2])
                price_prev_day =
float(close.iloc[-2])
                dollar_volume_prev =
vol_prev_day * price_prev_day
                if dollar_volume_prev > 0:

```

```

        pct_change =
(dollar_volume - dollar_volume_prev) /
dollar_volume_prev * 100
        if dollar_volume_prev
>= 1_000_000_000:
            prev_str =
f"${dollar_volume_prev/1_000_000_000:.2f}
Milyar"
            elif dollar_volume_prev
>= 1_000_000:
                prev_str =
f"${dollar_volume_prev/1_000_000:.2f}
Milyon"
            else:
                prev_str =
f"${dollar_volume_prev:,.0f}"
                arrow = "📈" if
pct_change >= 0 else "📉"
                sign = "+" if
pct_change >= 0 else ""
                prev_dollar_change_text
= f"\n    {arrow} Önceki Güne Göre:
{prev_str} → {sign}{pct_change:.1f}%"
            except:
                pass

        dollar_vol_line = (
            f"💰 İşlem Hacmi:
{vol_now:,.0f} adet × ${price:.2f} ≈
{dollar_vol_str}"
            f"{prev_dollar_change_text}"
        )

        strong_trend = adx_now >= 25

```

```

        adx_text = (
            f"✅ {adx_now:.0f} / Ref: 25
            üstü → Güçlü Trend" if strong_trend
            else f"⚠️ {adx_now:.0f} / Ref:
            25 üstü → Zayıf Trend"
        )

        index_perf =
        get_qqq_performance(days=20)
        rs_text = "— Veri yok"
        rs_strong = False
        rs_emoji = "📈"
        try:
            old_price =
            float(close.iloc[max(0,len(close)-20)])
            stock_perf = (price -
            old_price) / old_price * 100
            if index_perf is not None:
                rs_diff = stock_perf -
            index_perf

            rs_strong = rs_diff > 0
            rs_emoji = "📈" if
            rs_strong else "📉"
            rs_text = (
                f"✅ +{rs_diff:.1f}% /
                Ref: 0% üstü → Piyasayı Geride Bıraktı" if
                rs_strong

                else f"🛑
                {rs_diff:.1f}% / Ref: 0% üstü → Piyasanın
                Altında Kaldı"
            )
        except:
            pass

```



```

        divergence_text = "— Tespit
edilmedi"
        try:
            prices_10 = close.values[-10:]
            rsi_10 = rsi.values[-10:]
            p_low_idx =
int(np.argmin(prices_10[:-1]))
            p_high_idx =
int(np.argmax(prices_10[:-1]))
            bull_div = prices_10[-1] <
prices_10[p_low_idx] and rsi_10[-1] >
rsi_10[p_low_idx]
            bear_div = prices_10[-1] >
prices_10[p_high_idx] and rsi_10[-1] <
rsi_10[p_high_idx]
            if bull_div:
                divergence_text = "🐂 Boğa
Uyumsuzluğu"
            elif bear_div:
                divergence_text = "🐻 Ayı
Uyumsuzluğu"
        except:
            pass

        score = 0
        if price > sma200_now: score += 2
        if price > sma50_now: score += 1
        if price > ema20_now: score += 1
        if macd_crossover: score += 3
        elif macd_positive: score += 1
        if rsi_signal: score += 2
        if vol_ok: score += 2
        if strong_trend: score += 2
        if rs_strong: score += 1

```

```

if price < sma200_now: score -= 2
if resistance_risk: score -= 2
if not strong_trend: score -= 1

if score < 7:
    risk_score_i = max(10, min(90,
100 - (score * 8)))
    risk_label_i = "Düşük 🟢" if
risk_score_i < 30 else "Orta 🟡" if
risk_score_i < 60 else "Yüksek 🔴"
    vol_text_i = (
        f"✅ Bugün, 20 Günlük
Ort.'nın {vol_ratio:.1f} Katı. Ref: en az
1.5 kat → Yeterli" if vol_ok
        else f"🚫 Bugün, 20 Günlük
Ort.'nın {vol_ratio:.1f} Katı. Ref: en az
1.5 kat → Yetersiz"
    )
    macd_text_i = (
        f"✅ Taze Crossover 🔥 →
Pozitif (İyi)" if macd_crossover
        else f"✅ Pozitif (İyi)" if
macd_positive
        else f"🚫 Negatif"
    )
    rsi_text_i = (
        f"📊 {rsi_now:.1f} / Ref:
40-65 arası → Sağlıklı Bölgede" if
rsi_signal
        else f"📉 {rsi_now:.1f} /
Ref: 40-65 arası → Bölge Dışı"
    )
    low_data_note_i = f"\n⚠️ Not:
SMA200 güvenilir kaynaktan (Barchart)

```

```
alınamadı, sınırlı veriden ({data_days}
gün) tahmin edildi. Bu değer hatalı
olabilir.\n" if low_data_warning else ""
```

```
        yorum_parcalari = []
        yorum_parcalari.append("fiyat
SMA200'ün üzerinde" if price > sma200_now
else "fiyat SMA200'ün altında")
        yorum_parcalari.append("trend
gücü yeterli (ADX 25 üstü)" if strong_trend
else "trend gücü zayıf (ADX 25 altı)")
        yorum_parcalari.append("MACD
pozitif" if macd_positive else "MACD
negatif")
        yorum_parcalari.append("hacim
teyidi var" if vol_ok else "hacim teyidi
yok")

        ozet_i = f"Toplam skor
{score}/14 (eşik: 7+) altında kaldı. " + ",
".join(yorum_parcalari) + ". Net bir TREND
SİNYALİ için daha fazla kriterin aynı yönde
hizalanması gerekiyor."
```

```
msg_info = f"""
```

```
 TREND SİNYALİ – Bilgi Amaçlı (Eşik
Karşılanmadı)
 {symbol}
```

```
 Fiyat: ${price:.2f}
```

```
 Teknik Durum:
≈ SMA200 (Uzun Vadeli): {trend_sma200}
≈ EMA20 (Kısa Vadeli): {trend_ema20}
≈ SMA50 (Orta Vadeli): {trend_sma50}
```

 RSI (Momentum): {rsi_text_i}
 MACD (Yön Sinyali): {macd_text_i}

 Hacim (Adet Oranı): {vol_text_i}

{dollar_vol_line}

 ADX (Trend Gücü Endeksi): {adx_text}
{rs_emoji} Piyasaya Göre Güç (QQQ'a
Kıyasla): {rs_text}

 Uyumsuzluk (Fiyat-RSI Çelişkisi):
{divergence_text}

 Skor: {score}/14 (Sinyal eşiği:
7+)

 Risk Skoru: %{risk_score_i} -
{risk_label_i}
{low_data_note_i}

 Özet: {ozet_i}

{get_ohlc_source_label()}

 {now_tr().strftime('%d.%m.%Y %H:%M')}
""

return {"signal": None, "info":
msg_info}

score_for_signal = score

risk_score = max(10, min(90, 100 -
(score * 8)))

risk_label = "Düşük 🟢" if
risk_score < 30 else "Orta 🟡" if

```

risk_score < 60 else "Yüksek 🚫"

    entry = price
    take_profit = round(entry * 1.05,
2)
    atr_stop = round(entry - (atr_now *
1.8), 2)
    pct_stop = round(entry * 0.97, 2)
    stop_loss = round(max(atr_stop,
pct_stop * 0.98), 2)
    if stop_loss >= entry:
        stop_loss = round(entry * 0.97,
2)
    risk_amt = round(entry - stop_loss,
2)
    reward_amt = round(take_profit -
entry, 2)
    rr = round(reward_amt / risk_amt,
1) if risk_amt > 0 else 0

    vol_text = (
        f"✅ Bugün, 20 Günlük Ort.'nın
{vol_ratio:.1f} Katı 🔥. Ref: en az 1.5 kat
→ Yeterli" if vol_ratio >= 2
        else f"✅ Bugün, 20 Günlük
Ort.'nın {vol_ratio:.1f} Katı. Ref: en az
1.5 kat → Yeterli" if vol_ratio >= 1.5
        else f"🚫 Bugün, 20 Günlük
Ort.'nın {vol_ratio:.1f} Katı. Ref: en az
1.5 kat → Yetersiz"
    )
    macd_text = (
        f"✅ Taze Crossover 🔥 →
Pozitif (İyi)" if macd_crossover

```

```

        else "✅ Pozitif (İyi)" if
macd_positive
        else "🚫 Negatif"
    )
    rsi_text = (
        f"📊 {rsi_now:.1f} / Ref: 40-65
arası → Sağlıklı Bölgede" if rsi_signal
        else f"📉 {rsi_now:.1f} / Ref:
40-65 arası → Bölge Dışı"
    )

    if score >= 9 and rr >= 1.5:
        karar = "💚 İŞLEME GİRİLEBİLİR"
    elif score >= 7:
        karar = "🟡 İZLEMEDE KALSIN"
    else:
        karar = "🔴 GEÇ"

    sector, sector_count =
register_sector_signal(symbol)
    sector_text = f"🔥 {sector}
sektöründen {sector_count}. sinyal" if
sector_count >= 2 else f"📌 {sector}
sektörü"

    low_data_note = f"\n⚠️ Not: SMA200
güvenilir kaynaktan (Barchart) alınamadı,
sınırlı veriden ({data_days} gün) tahmin
edildi. Bu değer hatalı olabilir.\n" if
low_data_warning else ""

    msg = f"""
📊 <b>TREND SİNYALİ – Mevcut Güçlü
Trend</b>
📌 <b>{symbol} - POTANSİYEL SİNYAL</b>

```

 Sektör: {sector_text}

 Giriş: \${entry:.2f}

 Kar Al (%5): \${take_profit:.2f}

 Zarar Kes (ATR): \${stop_loss:.2f}

 Risk/Ödül: 1:{rr}

 Teknik Durum:

≈ SMA200 (Uzun Vadeli): {trend_sma200}

≈ EMA20 (Kısa Vadeli): {trend_ema20}

≈ SMA50 (Orta Vadeli): {trend_sma50}

 RSI (Momentum): {rsi_text}

 MACD (Yön Sinyali): {macd_text}

 Hacim (Adet Oranı): {vol_text}

{dollar_vol_line}

 ADX (Trend Gücü Endeksi): {adx_text}

{rs_emoji} Piyasaya Göre Güç (QQQ'a

Kıyasla): {rs_text}

 Uyumsuzluk (Fiyat-RSI Çelişkisi):

{divergence_text}

{sd_baslik}: {sd_text}

 Risk Skoru: %{risk_score} - {risk_label}

{low_data_note}

{get_ohlcv_source_label()}

 Karar: {karar}

 {now_tr().strftime('%d.%m.%Y %H:%M')}

"" ""

```

        return {"signal": msg, "info":
None}
    except Exception as e:
        import traceback
        print(f"[DEBUG analyze_stock]
{symbol}: {e}")
        traceback.print_exc()
        return None

# =====
# ERKEN UYARI SİSTEMİ (7 Maddelik
Uyumsuzluk Teyidi)
# =====

def early_warning_scan(symbol, df=None):
    """
        Sadece RSI/fiyat uyumsuzluğuna dayanan
        erken dönüş sinyali.
        7 kriterle teyit edilir, ana trend
        filtresinden bağımsız çalışır.
    """
    try:
        if df is None:
            df = td_get_ohlc(symbol,
outputsize=60)
        if df is None:
            print(f"[DEBUG
early_warning_scan] {symbol}: df is None
(veri çekilemedi)")
            return None
        if len(df) < 30:
            print(f"[DEBUG
early_warning_scan] {symbol}: len<30 (len=
{len(df)}), muhtemel yeni IPO")

```



```

        return {"signal": None, "info":
None, "ipo": True, "ipo_message":
ipo_short_data_message(symbol, len(df),
30)}

        close = df['Close']
        volume = df['Volume']
        rsi = calc_rsi(close)
        macd, macd_signal =
calc_macd(close)
        sma200 = calc_sma(close, min(200,
len(close)-1))
        ema20 = calc_ema(close, 20)
        vol_ma20 =
volume.rolling(20).mean()

        prices_10 = close.values[-10:]
        rsi_10 = rsi.values[-10:]
        p_low_idx =
int(np.argmin(prices_10[:-1]))
        p_high_idx =
int(np.argmax(prices_10[:-1]))

        bull_div = prices_10[-1] <
prices_10[p_low_idx] and rsi_10[-1] >
rsi_10[p_low_idx]
        bear_div = prices_10[-1] >
prices_10[p_high_idx] and rsi_10[-1] <
rsi_10[p_high_idx]

        if not (bull_div or bear_div):
            price_i = float(close.iloc[-1])
            rsi_i = float(rsi.iloc[-1])
            macd_i = float(macd.iloc[-1])

```

```

        macd_sig_i =
float(macd_signal.iloc[-1])
        msg_info = f"""
⚡ <b>ERKEN UYARI – Bilgi Amaçlı
(Uyumsuzluk Yok)</b>
🦊 <b>{symbol}</b>

$ Fiyat: ${price_i:.2f}
📈 RSI(14): {rsi_i:.1f}
📊 MACD: {macd_i:.3f} (Sinyal:
{macd_sig_i:.3f}) – {"✅ Pozitif" if macd_i
> macd_sig_i else "⚠️ Negatif"}

💬 <b>Özet:</b> Son 10 günde fiyat ile RSI
arasında bir uyumsuzluk (divergence) tespit
edilmedi. Erken uyarı sistemi yalnızca
uyumsuzluk durumunda kriter taraması
başlatır, bu yüzden teyit kriterleri
hesaplanmadı.

{get_ohlcv_source_label()}
🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}
"""

        return {"signal": None, "info":
msg_info}

        direction = "BOĞA" if bull_div else
"AYI"
        price = float(close.iloc[-1])

        # KRİTER 1: Hacim teyidi (adet
bazlı oran – skora/sayaca katkı sağlar)
        vol_now = float(volume.iloc[-1])
        vol_avg = float(vol_ma20.iloc[-1])

```

```

if not pd.isna(vol_ma20.iloc[-1]) else
vol_now
    vol_ratio = vol_now / vol_avg if
vol_avg > 0 else 0
    crit_volume = vol_ratio >= 1.25
    vol_kriter_text = (
        f"{'✅' if crit_volume else
'❌'} Bugün, 20 Günlük Ort.'nın
{vol_ratio:.1f} Katı. Ref: en az 1.25 kat
→ {'Yeterli' if crit_volume else
'Yetersiz'}"
    )

    # 💰 Dolar bazlı işlem hacmi (Adet
× Fiyat) – SADECE BİLGİ AMAÇLI, kriter
sayacına katkısı YOK.
    # Mutlak bir referans yerine ÖNCEKİ
GÜNE göre karşılaştırma gösterilir.
    dollar_volume = vol_now * price
    if dollar_volume >= 1_000_000_000:
        dollar_vol_str =
f"${dollar_volume/1_000_000_000:.2f}
Milyar"
    elif dollar_volume >= 1_000_000:
        dollar_vol_str =
f"${dollar_volume/1_000_000:.2f} Milyon"
    else:
        dollar_vol_str =
f"${dollar_volume:,.0f}"

    prev_dollar_change_text = ""
    try:
        if len(df) >= 2:
            vol_prev_day =

```

```

float(volume.iloc[-2])
        price_prev_day =
float(close.iloc[-2])
        dollar_volume_prev =
vol_prev_day * price_prev_day
        if dollar_volume_prev > 0:
            pct_change =
(dollar_volume - dollar_volume_prev) /
dollar_volume_prev * 100
            if dollar_volume_prev
>= 1_000_000_000:
                prev_str =
f"${dollar_volume_prev/1_000_000_000:.2f}
Milyar"
            elif dollar_volume_prev
>= 1_000_000:
                prev_str =
f"${dollar_volume_prev/1_000_000:.2f}
Milyon"
            else:
                prev_str =
f"${dollar_volume_prev:,.0f}"
                arrow = "📈" if
pct_change >= 0 else "📉"
                sign = "+" if
pct_change >= 0 else ""
                prev_dollar_change_text
= f"\n {arrow} Önceki Güne Göre:
{prev_str} → {sign}{pct_change:.1f}%"
            except:
                pass

dollar_vol_line = (
    f"💰 İşlem Hacmi:

```

```

{vol_now:,.0f} adet × ${price:.2f} ≈
{dollar_vol_str}"
        f"{prev_dollar_change_text}"
    )

    # KRİTER 2: Çoklu zaman dilimi
    (haftalık RSI)
    weekly_rsi = get_weekly_rsi(symbol,
df=df)
    if direction == "BOĞA":
        crit_timeframe = weekly_rsi is
not None and weekly_rsi < 55
        timeframe_text = (
            f"{'✅' if crit_timeframe
else '❌'} {weekly_rsi:.1f} / Ref: 55 altı
(Boğa) → {'Destekliyor' if crit_timeframe
else 'Desteklemiyor'}"
            if weekly_rsi is not None
else "❌ Veri yok"
        )
    else:
        crit_timeframe = weekly_rsi is
not None and weekly_rsi > 45
        timeframe_text = (
            f"{'✅' if crit_timeframe
else '❌'} {weekly_rsi:.1f} / Ref: 45 üstü
(Ayı) → {'Destekliyor' if crit_timeframe
else 'Desteklemiyor'}"
            if weekly_rsi is not None
else "❌ Veri yok"
        )

    # KRİTER 3: Destek/Direnç seviyesi
    çıkışması

```

```

# Fiyat ortalamanın ÜSTÜNDEYSE o
ortalama DESTEK, ALTINDAYSA DİRENÇ sayılır.
# SMA200: Veri yetersizse (örn.
stockanalysis.com'un ~50 günlük limiti),
# Barchart.com'un hazır hesapladığı
SMA200 değerine düş.
if len(df) >= 190:
    sma200_now =
float(sma200.iloc[-1]) if not
pd.isna(sma200.iloc[-1]) else price
else:
    barchart_sma200 =
get_sma200_barchart(symbol)
    sma200_now = barchart_sma200 if
barchart_sma200 is not None else (
        float(sma200.iloc[-1]) if
not pd.isna(sma200.iloc[-1]) else price
    )
    ema20_now = float(ema20.iloc[-1])
    dist_sma200 = abs(price -
sma200_now) / price * 100
    dist_ema20 = abs(price - ema20_now)
/ price * 100
    sma200_label = "Destek" if price >=
sma200_now else "Direnç"
    ema20_label = "Destek" if price >=
ema20_now else "Direnç"
    near_support = dist_sma200 < 3 or
dist_ema20 < 2
    crit_support = near_support
    support_text = (
        f"{'✅' if crit_support else
'❌'} SMA200 [{sma200_label}] %
{dist_sma200:.1f} uzakta / "

```

```

        f"EMA20 [{ema20_label}] %
{dist_ema20:.1f} uzakta → {'Yakın (İyi)'
if crit_support else 'Uzak'}"
    )

    # KRİTER 4: Mum formasyonu
    candle_pattern =
detect_candle_pattern(df)
    crit_candle = candle_pattern is not
None
    candle_text = f"✅ {candle_pattern}
tespit edildi" if crit_candle else "❌
Tespit edilmedi"

    # KRİTER 5: MACD histogram
daralması
    macd_hist = (macd -
macd_signal).values[-5:]
    if direction == "BOĞA":
        crit_macd_hist = macd_hist[-1]
> macd_hist[-3]
    else:
        crit_macd_hist = macd_hist[-1]
< macd_hist[-3]
    macd_hist_text = (
        f"{'✅' if crit_macd_hist else
'❌'} Son {macd_hist[-1]:.3f} / Önceki
{macd_hist[-3]:.3f} "
        f"→ {'Momentum Yönünde (İyi)'
if crit_macd_hist else 'Henüz Değişmedi'}"
    )

    # KRİTER 6: Piyasa/sektör genel
durumu (QQQ) – eşik %-2

```

```

        qqk_perf =
get_qqq_performance(days=5)
        qqk_emoji = "📈" if (qqk_perf is
not None and qqk_perf >= 0) else "📉"
        if direction == "BOĞA":
            crit_market = qqk_perf is not
None and qqk_perf > -2
            market_text = (
                f"{'✅' if crit_market else
'❌'} QQK %{qqk_perf:.1f} / Ref: -2% üstü
(Boğa) → {'Uygun' if crit_market else
'Uygun Değil'}"
                if qqk_perf is not None
else "❌ Veri yok"
            )
        else:
            crit_market = qqk_perf is not
None and qqk_perf < 2
            market_text = (
                f"{'✅' if crit_market else
'❌'} QQK %{qqk_perf:.1f} / Ref: 2% altı
(Ayı) → {'Uygun' if crit_market else
'Uygun Değil'}"
                if qqk_perf is not None
else "❌ Veri yok"
            )

        # KRİTER 7: Aşırı satım/alım
derinliği
        rsi_now = float(rsi.iloc[-1])
        if direction == "BOĞA":
            crit_depth = rsi_now < 35
            depth_emoji = "❄️" if
crit_depth else "🌱"

```



```

        depth_text = f"{'✅' if
crit_depth else '❌'} {rsi_now:.1f} / Ref:
35 altı (Boğa) → {'Yeterince Düşük' if
crit_depth else 'Henüz Yeterince Düşük
Değil'}"
    else:
        crit_depth = rsi_now > 65
        depth_emoji = "🔥" if
crit_depth else "🧊"
        depth_text = f"{'✅' if
crit_depth else '❌'} {rsi_now:.1f} / Ref:
65 üstü (Ayı) → {'Yeterince Yüksek' if
crit_depth else 'Henüz Yeterince Yüksek
Değil'}"

    criteria = [crit_volume,
crit_timeframe, crit_support, crit_candle,
crit_macd_hist, crit_market, crit_depth]
    confirmed = sum(criteria)

    if confirmed < 3:
        confidence_i = "Düşük 🟡" if
confirmed == 2 else "Çok Düşük 🔴"
        yorum_parcalari = []
        yorum_parcalari.append("hacim
teyidi var" if crit_volume else "hacim
teyidi yok")

        yorum_parcalari.append("haftalık zaman
dilimi destekliyor" if crit_timeframe else
"haftalık zaman dilimi desteklemiyor")
        yorum_parcalari.append("RSI
yeterince aşırı bölgede" if crit_depth else
"RSI henüz aşırı bölgede değil")

```

```
ozet_i = f"{direction}
uyumsuzluğu tespit edildi ama yalnızca
{confirmed}/7 kriter karşılandı (gerekli:
3+). " + ", ".join(yorum_parcalari) + "."
```

```
msg_info = f"""
⚡ <b>ERKEN UYARI – Bilgi Amaçlı (Eşik
Karşılanmadı)</b>
🦅 <b>{symbol} - {direction} UYUMSUZLUĞU
(zayıf)</b>
```

```
$ Fiyat: ${price:.2f}
{dollar_vol_line}
```

```
📄 <b>Teyit Kriterleri ({confirmed}/7):</b>
🌐 Hacim Teyidi (Adet Oranı):
{vol_kriter_text}
📅 Haftalık Teyit (Büyük Resim RSI):
{timeframe_text}
🛡️ Destek/Direnç (Ref Yakınlığı):
{support_text}
🕯️ Mum Formasyonu: {candle_text}
⇒ MACD Daralması (Momentum Değişimi):
{macd_hist_text}
{qqq_emoji} Piyasa Durumu (Genel Piyasa
QQQ): {market_text}
{depth_emoji} Aşırı Satım/Alım (RSI
Derinliği): {depth_text}
```

```
🎯 Güven Seviyesi: <b>{confidence_i}</b>
(eşik altı)
```

```
💬 <b>Özet:</b> {ozet_i}
```

```
{get_ohlc_source_label()}
🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}
```

⚠ Bu bilgi amaçlıdır, sinyal eşiği (3/7) karşılanmamıştır.

```
"""
        return {"signal": None, "info":
msg_info}

        confidence = "Yüksek 🟢" if
confirmed >= 5 else "Orta 🟡" if confirmed
>= 4 else "Düşük 🟠"

        yon_emoji = "🐂" if direction ==
"BOĞA" else "🐻"
        emoji = "🟢" if direction == "BOĞA"
else "🔴"
        action = "Yukarı dönüş potansiyeli"
if direction == "BOĞA" else "Aşağı dönüş
riski"

        msg = f"""
⚡ <b>ERKEN UYARI – Dönüş Potansiyeli</b>
📌 <b>{symbol} - {yon_emoji} {direction}
UYUMSUZLUĞU</b>

        {emoji} {action}
💰 Fiyat: ${price:.2f}
{dollar_vol_line}

📋 <b>Teyit Kriterleri ({confirmed}/7):</b>
🌐 Hacim Teyidi (Adet Oranı):
{vol_kriter_text}
📅 Haftalık Teyit (Büyük Resim RSI):
```

```

{timeframe_text}
🛡 Destek/Direnç (Ref Yakınlığı):
{support_text}
🕯 Mum Formasyonu: {candle_text}
⇒ MACD Daralması (Momentum Değişimi):
{macd_hist_text}
{qqq_emoji} Piyasa Durumu (Genel Piyasa
QQQ): {market_text}
{depth_emoji} Aşırı Satım/Alım (RSI
Derinliği): {depth_text}

🎯 Güven Seviyesi: <b>{confidence}</b>

{get_ohlc_source_label()}
🕒 {now_tr().strftime('%d.%m.%Y %H:%M')}

⚠ Bu erken bir sinyal, ana trend
filtresinden geçmemiştir. Dikkatli
değerlendir.
"""

        return {"signal": msg, "info":
None}
    except Exception as e:
        import traceback
        print(f"[DEBUG early_warning_scan]
{symbol}: {e}")
        traceback.print_exc()
        return None

# =====
# HABERLER
# =====

def is_important_news(title, summary=""):

```

```
"""
```

Haber başlığı ve özetine bakarak
haber önemli olup olmadığını filtreler.

Küçük/rutin haberler elenerek sadece
piyasayı gerçekten etkileyebilecek
büyük gelişmeler geçirilir.

Önemli haber kategorileri: kazanç/gelir
açıklaması, CEO/yönetici değişikliği,
birleşme/satın alma, dava/soruşturma,
FDA onayı/reddi, büyük sözleşme,
iflas/borç, hisse geri alımı/temettü,
not/hedef fiyat değişikliği (büyük).

```
"""
```

```
text = (title + " " + summary).lower()
important_keywords = [
    'earnings', 'revenue', 'profit',
    'loss', 'beat', 'miss', 'guidance',
    'quarterly', 'annual', 'eps', 'q1',
    'q2', 'q3', 'q4',
    'ceo', 'cfo', 'executive',
    'resign', 'appoint', 'leadership',
    'merger', 'acquisition', 'acquire',
    'buyout', 'takeover', 'deal',
    'lawsuit', 'sue', 'investigation',
    'sec', 'ftc', 'doj', 'fine', 'penalty',
    'fda', 'approval', 'approved',
    'reject', 'clinical',
    'contract', 'partnership',
    'agreement',
    'bankruptcy', 'debt', 'layoff',
    'restructur',
    'buyback', 'dividend', 'split',
    'upgrade', 'downgrade', 'target',
```

```

'overweight', 'underweight',
    'recall', 'warning', 'shortage',
'outage',
    'ipo', 'offering', 'dilut',
]
return any(kw in text for kw in
important_keywords)

def translate_and_interpret_news(title,
summary, symbol):
    """
        Claude API (Haiku modeli) kullanarak
haber başlığını ve özetini
        Türkçe'ye çevirir ve kısa bir sentiment
yorumu ekler.

        Maliyet tahmini: ~300 input + 200
output token/haber
        Aylık ~110 haber için: ~$3-5 (Haiku)

        Eğer API çağrısı başarısız olursa (rate
limit, bağlantı hatası vb.)
        orijinal İngilizce metni döndürür –
sessizce başarısız olur.
    """
    try:
        if not ANTHROPIC_API_KEY:
            print("[DEBUG
translate_and_interpret_news]
ANTHROPIC_API_KEY tanımlı değil, İngilizce
fallback")
            return f"{title}\n{summary}"

        payload = {

```

```
"model": "claude-haiku-4-5-20251001",
    "max_tokens": 300,
    "messages": [{
        "role": "user",
        "content": (
            f"Aşağıdaki haber
başlığını ve özetini Türkçe'ye çevir, "
            f"ardından bu haberin
{symbol} hissesi için pozitif mi, negatif
mi "
            f"yoksa nötr mü
olduğunu tek bir emoji ile belirt "
            f"(🟢 pozitif / 🟡
negatif / ⚪ nötr). "
            f"Format: [Çevrilmiş
Başlık]\n[Çevrilmiş Özet]\n[Emoji] [Kısa
yorum - max 1 cümle]\n\n"
            f"Başlık:
{title}\nÖzet: {summary}"
        )
    }]
}
r = requests.post(

"https://api.anthropic.com/v1/messages",
    headers={
        "Content-Type":
"application/json",
        "x-api-key":
ANTHROPIC_API_KEY,
        "anthropic-version": "2023-
06-01"
    },
```

```

        json=payload,
        timeout=15
    )
    if r.status_code == 200:
        data = r.json()
        text = data.get("content",
[{}])[0].get("text", "")
        if text:
            return text.strip()
        else:
            print(f"[DEBUG
translate_and_interpret_news] API hatası:
{r.status_code} {r.text[:200]}")
    except Exception as e:
        print(f"[DEBUG
translate_and_interpret_news] Exception:
{e}")
        # Fallback: orijinal İngilizce
        return f"{title}\n{summary}"

def fetch_news_for_symbol(symbol,
importance_filter=True, days_back=3):
    """
    Finnhub /company-news API'sinden sembol
    bazlı haberleri çeker
    (stockanalysis.com scraping yerine –
    daha güvenilir, JSON tabanlı).

    importance_filter=True olduğunda sadece
    önemli haberler döner
    (kazanç, CEO değişikliği, birleşme,
    dava vb.).
    importance_filter=False olduğunda tüm
    haberler döner.

```



```

"""
    if not FINNHUB_API_KEY:
        print("[DEBUG
fetch_news_for_symbol] FINNHUB_API_KEY
tanımlı değil")
        return []
    try:
        to_date = now_tr().date()
        from_date = to_date -
timedelta(days=days_back)
        r = requests.get(

"https://finnhub.io/api/v1/company-news",
        params={
            "symbol": symbol.upper(),
            "from":
from_date.isoformat(),
            "to": to_date.isoformat(),
            "token": FINNHUB_API_KEY
        },
        timeout=12
        )
        if r.status_code != 200:
            print(f"[DEBUG
fetch_news_for_symbol] {symbol}: HTTP
{r.status_code}")
            return []
        items = r.json()
        if not isinstance(items, list):
            return []
        results = []
        for item in items:
            title = (item.get("headline")
or "").strip()

```

```

        summary = (item.get("summary")
or "").strip()
        if not title:
            continue
        if importance_filter and not
is_important_news(title, summary):
            continue
        ts = item.get("datetime")
        date_str =
datetime.fromtimestamp(ts,
tz=timezone.utc).astimezone(TR_TZ).strftime
("%d.%m %H:%M") if ts else ""
        results.append({
            "title": title,
            "summary": summary,
            "link": (item.get("url") or
"".strip(),
            "date": date_str,
            "id": item.get("id") or f"
{symbol}:{title}",
        })
        if len(results) >= 5:
            break
        return results
    except Exception as e:
        print(f"[DEBUG
fetch_news_for_symbol] {symbol}: {e}")
        return []

def
fetch_finnhub_general_news(importance_filte
r=True, max_items=15):
    """

```

```
Finnhub /news?category=general
API'sinden piyasa geneli (sembolle bağlı
    olmayan) haberleri çeker. Kanal 5'in
bağımsız periyodik taraması için.
"""
    if not FINNHUB_API_KEY:
        print("[DEBUG
fetch_finnhub_general_news] FINNHUB_API_KEY
tanımlı değil")
        return []
    try:
        r = requests.get(

"https://finnhub.io/api/v1/news",
            params={"category": "general",
"token": FINNHUB_API_KEY},
            timeout=12
        )
        if r.status_code != 200:
            print(f"[DEBUG
fetch_finnhub_general_news] HTTP
{r.status_code}")
            return []
        items = r.json()
        if not isinstance(items, list):
            return []
        results = []
        for item in items:
            title = (item.get("headline")
or "").strip()
            summary = (item.get("summary")
or "").strip()
            if not title:
                continue
```

```

        if importance_filter and not
is_important_news(title, summary):
            continue
        results.append({
            "title": title,
            "summary": summary,
            "link": (item.get("url") or
"".strip(),
            "related":
(item.get("related") or "").strip(),
            "id": item.get("id") or
title,
        })
        if len(results) >= max_items:
            break
        return results
    except Exception as e:
        print(f"[DEBUG
fetch_finnhub_general_news] {e}")
        return []

def send_news_for_signal(symbol,
source_label=""):
    """
        Kanal 1/2/3/4'ten biri sinyal
        ürettiğinde çağrılır. 0 sembolle ilgili
        önemli haberleri Finnhub'dan çekip
        Türkçeye çevirip Kanal 5'e gönderir.
        Aynı haber tekrar gönderilmez
        (sent_news_set kontrolü).
    """
    try:
        news_items =

```

```

fetch_news_for_symbol(symbol,
importance_filter=True)
    for item in news_items[:3]:
        nid = item["id"]
        if is_news_already_sent(nid):
            continue
        translated =
translate_and_interpret_news(item["title"],
item["summary"], symbol)
        baslik = f"📰 <b>{symbol} –
İlgili Haber</b>" + (f" ({source_label})"
if source_label else "")
        msg = f"{baslik}\n{translated}"
        if item.get("link"):
            msg += f"\n🔗
{item['link']}"
            send_kanal(msg, "haber")
            mark_news_sent(nid)
    except Exception as e:
        print(f"[DEBUG
send_news_for_signal] {symbol}: {e}")

# =====
# TÜRKÇE NORMALİZASYON
# =====

def normalize_text(text):
    rep =
{'İ': 'i', 'I': 'i', 'ı': 'i', 'Ü': 'u', 'ü': 'u', 'Ö':
': 'o', 'ö': 'o', 'Ş': 's', 'ş': 's', 'Ğ': 'g', 'ğ': '
g', 'Ç': 'c', 'ç': 'c', 'Â': 'a', 'â': 'a'}
    text = text.lower()
    for k,v in rep.items():
        text =

```

```

text.replace(k.lower(),v).replace(k,v)
    return text

# =====
# KOMUT İŞLEYİCİ
# =====

def handle_command(text, chat_id,
thread_id=None):
    # Bu komut hangi konudan (topic)
    geldiyse, aynı fonksiyon içindeki TÜM
    # send_telegram(msg, chat_id) çağrılarını
    otomatik olarak o konuya cevap
    # versin diye send_telegram'ı bu scope
    içinde gölgeliyoruz. Başka bir
    # chat_id'ye açıkça gönderilen mesajlar
    (örn. broadcast) etkilenmez.
    _global_send_telegram = send_telegram
    def send_telegram(msg, cid=None,
    tid=None):
        if cid is None or cid == chat_id:
            return
        _global_send_telegram(msg, chat_id, tid if
        tid is not None else thread_id)
        return _global_send_telegram(msg,
        cid, tid)

    t = normalize_text(text.strip())

    # BOT AÇMA/KAPATMA – diğer her komuttan
    önce kontrol edilir, /kapat
    # sırasında bile çalışır.
    normalize_text zaten büyük/küçük harf ve
    # Türkçe karakter farkını çözüyor (örn.

```

```

"/AÇ" -> "/ac").
    OPEN_WORDS =
['/ac', '/ack', '/acik', '/open', '/start_bot',
'/basla', '/turnon', '/aktif']
    CLOSE_WORDS =
['/kapat', '/kapa', '/off', '/turnoff', '/bitir',
'/passive', '/sonlandir', '/durdur']
    if any(t.startswith(x) for x in
OPEN_WORDS):
        bot_manual_state["active"] = True
        send_telegram("✅ Bot manuel olarak
AÇILDI. Otomatik tarama, piyasa saatlerine
göre tekrar çalışacak.", chat_id)
        return
    if any(t.startswith(x) for x in
CLOSE_WORDS):
        bot_manual_state["active"] = False
        send_telegram("🚫 Bot manuel olarak
KAPATILDI. Otomatik tarama durdu,
kendiliğinden mesaj göndermeyecek.\nManuel
komutlar (/analizet, /dalga, /formasyon
vb.) hâlâ çalışır.\nTekrar açmak için: /ac
(veya /başla, /turnon)", chat_id)
        return

    if any(t.startswith(x) for x in
['/start', '/durum', '/status']):
        send_telegram(f""🦅 <b>HAWK SIGNAL
BOT – NASDAQ</b>

/durum – /status
/yardim – /help
/liste – /list

```

📊 TARAMA | ⚡ ERKEN UYARI
🌐 ELLİOTT DALGA | 📈 FORMASYON
📌 TAKİP | 🇺🇸 ANALİZ | 📰 HABER
🔔 ALARMLAR | 📁 PORTFÖY

🤖 Bot Durumu: {"✅ Açık" if
bot_manual_state["active"] else "🚫 Manuel
Kapalı"}

Detay için /yardim veya /help"", chat_id)
return

```
if any(t.startswith(x) for x in  
['/yardim', '/yardım', '/komutlar', '/komut'])  
:  
    send_telegram("📋 <b>KOMUTLAR –  
Hawk Signal (NASDAQ)</b>
```

🏠 GENEL
/start – Karşılama ve genel bilgi
/durum – Sistem durumu (eş değer: /status)
/liste – Havuz/tarama durumu (eş değer:
/list)
/yardim – Bu liste (eş değer: /yardım,
/komutlar, /komut)
/help – İngilizce komut listesi

🔊 BOT AÇ/KAPAT (sadece otomatik
taramayı durdurur, manuel komutlar her
zaman çalışır)
/ac – Botu aç (eş değer: /acik, /open,
/basla, /turnon, /aktif)
/kapat – Botu kapat (eş değer: /kapa, /off,
/turnoff, /bitir, /passive, /sonlandir,

/durdur)

 **TARAMA (Trend Sinyali)**

/nasdaq tara – Genel tarama (eş değer: /abd tara)

/nasdaq [HİSSE] – Tekil trend analizi

 **ERKEN UYARI (Dönüş Sinyali)**

/erkenuyari tara – Genel tarama (eş değer: /earlywarning)

/erkenuyari [HİSSE] – Tekil erken uyarı analizi

 **ELLIOTT DALGA SAYIMI**

/dalga [HİSSE] – Tekil dalga analizi (derin analiz, 5 dalga kontrolü)

/dalga tara – Manuel genel tarama (eş değer: /elliott, /eliot)

 Otomatik Tarama: Her 2.5 saatte bir, losers listesinin ilk 100 hissesini tarar; RSI/hacim/düşüş günü/destek yakınlığı kriterlerinden 3/4'ü karşılanırsa + ZigZag pivot dönüşü tespit edilirse bildirim verir.

 **FORMASYON SİNYALİ**

/formasyon [HİSSE] – Tekil formasyon analizi

/formasyon tara – Genel tarama (eş değer: /formation, /pattern)

Dönüş: Wolfe Wave, Wedge, H&S, Diamond, Three Drives

Devam: Triangle, Flag, Rectangle

 **ANALİZ** (Trend + Erken Uyarı birlikte, eşik karşılanmasa da güncel durumu gösterir)
/analizet [HİSSE]
(eş değer: /analiz, /analyze)

 **HABER**
/haber [HİSSE] – Sembolde özel önemli haberler (Türkçe çeviri + yorum)
(eş değer: /news)
🔊 Otomatik: Trend/Erken Uyarı sinyali geldiğinde ilgili önemli haberler de otomatik olarak eklenir (kazanç, CEO, birleşme vb.)

 **GÜNÜN KAZANANLARI**
/kazanalar – Günün en çok kazanan 50 hisse (tablo: Hisse | Kazanç% | Dolar Hacmi | Adet)
(eş değer: /gainers, /top)

 **TAKİP**
/takip [HİSSE] [GİRİŞ] [STOP] [HEDEF] – Yeni takip ekle
/takiplerim – Takip listesini göster
(eş değer: /track, /mytracks)

 **ALARMLAR**
/alarm [HİSSE] [FİYAT] – Yeni alarm kur
/alarmlarım – Aktif alarmları listele
/alarm sil [HİSSE] – Alarmı kaldır
(eş değer: /alert, /alerts, /alert delete)

 **PORTFÖY**

```
/portfoy ekle [HISSE] [FIYAT] [ADET] –  
Portföye ekle  
/portfoy [HISSE] – Tek hissenin durumu  
/portfoy – Tüm portföyü göster  
(eş değer: /portföy, /portfolio, /portfolio  
add)
```

 For the English command list, type
/help.""", chat_id)
return

```
if t.startswith('/help'):  
    send_telegram(""" <b>COMMANDS –  
Hawk Signal (NASDAQ)</b>
```

 GENERAL

```
/start – Welcome and general info  
/status – System status (alias: /durum)  
/list – Pool/scan status (alias: /liste)  
/help – This list  
/yardim – Turkish command list
```

 BOT ON/OFF (only stops automatic
scanning, manual commands always work)
/ac – Turn bot on (aliases: /acik, /open,
/basla, /turnon, /aktif)
/kapat – Turn bot off (aliases: /kapa,
/off, /turnoff, /bitir, /passive,
/sonlandir, /durdur)

 SCAN (Trend Signal)

```
/nasdaq scan – General scan (alias: /abd  
scan)
```

/nasdaq [TICKER] – Single trend analysis

⚡ **EARLY WARNING (Reversal Signal)**
/earlywarning scan – General scan (alias:
/erkenuyari)
/earlywarning [TICKER] – Single early
warning analysis

🌊 **ELLIOTT WAVE COUNT**
/elliott [TICKER] – Single wave analysis
(deep analysis, 5-wave check)
/elliott scan – Manual general scan
(aliases: /eliot, /dalga)
🖱️ Auto Scan: Every 2.5 hours, scans top
100 from the losers list;
if 3/4 of RSI/volume/down-day/support-
proximity criteria are met
AND a ZigZag pivot reversal is detected,
sends a notification.

📐 **FORMATION SIGNAL**
/formation [TICKER] – Single formation
analysis
/formation scan – General scan
(aliases: /formasyon, /pattern)
Reversal: Wolfe Wave, Wedge, H&S, Diamond,
Three Drives
Continuation: Triangle, Flag, Rectangle

📊 **ANALYSIS** (Trend + Early Warning
together, shows current status even below
threshold)
/analyze [TICKER]
(aliases: /analiz, /analizet)

 NEWS

/news [TICKER] – Symbol-specific important news (Turkish translation + sentiment)
(alias: /haber)

 Auto: When a Trend/Early Warning signal is generated, relevant important news (earnings, CEO, merger, etc.) is automatically attached.

 TOP GAINERS

/gainers – Today's top 50 gaining stocks
(table: Ticker | Gain% | Dollar Volume | Shares)
(aliases: /kazananlar, /top)

 TRACKING

/track [TICKER] [ENTRY] [STOP] [TARGET] – Add new tracking
/mytracks – Show tracking list
(aliases: /takip, /takiplerim)

 ALERTS

/alert [TICKER] [PRICE] – Set new alert
/alerts – List active alerts
/alert delete [TICKER] – Remove alert
(aliases: /alarm, /alarmlarim, /alarm sil)

 PORTFOLIO

/portfolio add [TICKER] [PRICE] [QTY] – Add to portfolio
/portfolio [TICKER] – Single ticker status
/portfolio – Show full portfolio
(aliases: /portfoy, /portföy, /portfoy)

```
ekle)
```

```
    ⓘ Türkçe komut listesi için /yardim  
    yazabilirsin.""", chat_id)  
    return
```

```
    if any(t.startswith(x) for x in  
    ['/liste', '/list']):  
        send_telegram(f"📄 <b>HAVUZ  
DURUMU</b>\n\n{get_source_label()}\nSon  
sayı: {last_pool_source['count']}  
hisse\nTarayıcı kütüphanesi: {'Aktif ✅' if  
TV_SCRAPER_AVAILABLE else 'Yok, yedek modda  
⚠️'}", chat_id)  
    return
```

```
    # TREND TARAMASI  
    if t.startswith('/nasdaq') or  
t.startswith('/abd'):  
        cmd_len = 7 if  
t.startswith('/nasdaq') else 4  
        rest = t[cmd_len:].strip()  
        if any(x in rest for x in  
['tara', 'scan']):  
            send_telegram("🔍 NASDAQ trend  
taraması başladı...", chat_id)  
            def do_scan(cid=chat_id):  
                reset_sector_tracking()  
                batch =  
get_trading_pool(30)  
                if  
last_pool_source["source"] == "memory":  
                    send_telegram("⚠️ Canlı
```

```

veri alınamadı, son başarılı tarama
hafızasına geçildi.", cid)
        elif
last_pool_source["source"] == "none":
            send_telegram("❌
Hiçbir veri kaynağına erişilemedi, tarama
yapılamadı.", cid)
            return
        found = 0
        for ticker in batch:
            result =
analyze_stock(ticker)
            if result and
result.get("signal"):

send_telegram(result["signal"], cid)
                found += 1
                time.sleep(2)
                time.sleep(1.2)
            if found == 0:
                send_telegram("Tarama
tamamlandı. Uygun trend sinyali
bulunamadı.", cid)

threading.Thread(target=do_scan).start()
        elif rest:
            ticker = rest.upper().split()
[0]
            send_telegram(f"🔍 {ticker}
analiz ediliyor...", chat_id)
            def do_single(tk=ticker,
cid=chat_id):
                result = analyze_stock(tk)
                if result and

```

```

result.get("signal"):

send_telegram(result["signal"], cid)
    elif result and
result.get("ipo"):

send_telegram(result["ipo_message"], cid)
    elif result and
result.get("info"):

send_telegram(result["info"], cid)
    else:
        send_telegram(f"{tk}
için veri alınamadı.", cid)

threading.Thread(target=do_single).start()
    return

# ERKEN UYARI
    if any(t.startswith(x) for x in
['/erkenuyari', '/erken
uyari', '/earlywarning']):
        rest = re.sub(r'^/(erkenuyari|erken
uyari|earlywarning)', '', t).strip()
        if any(x in rest for x in
['tara', 'scan']):
            send_telegram("⚡ Erken uyarı
taraması başladı (7 kriterli dönüş
teyidi)...", chat_id)
            def do_ew(cid=chat_id):
                batch =
get_trading_pool(25)
                found = 0
                for ticker in batch:

```



```

        result =
early_warning_scan(ticker)
        if result and
result.get("signal"):

send_telegram(result["signal"], cid)
        found += 1
        time.sleep(2)
        time.sleep(1.2)
        if found == 0:
            send_telegram("Erken
uyarı taraması tamamlandı. Teyitli dönüş
sinyali bulunamadı.", cid)

threading.Thread(target=do_ew).start()
        elif rest:
            ticker = rest.upper().split()
[0]
            send_telegram(f"⚡ {ticker}
erken uyarı analizi yapılıyor...", chat_id)
            def do_single_ew(tk=ticker,
cid=chat_id):
                result =
early_warning_scan(tk)
                if result and
result.get("signal"):

send_telegram(result["signal"], cid)
                elif result and
result.get("ipo"):

send_telegram(result["ipo_message"], cid)
                elif result and
result.get("info"):

```

```

send_telegram(result["info"], cid)
    else:
        send_telegram(f"{tk}
için veri alınamadı.", cid)

threading.Thread(target=do_single_ew).start
()

    return

    # ANALİZ
    if any(t.startswith(x) for x in
['/analizet', '/analiz', '/analyze']):
        parts = text.split(None,1)
        if len(parts) < 2:
            send_telegram("Kullanım:
/analizet [HİSSE]", chat_id)
            return
        ticker =
parts[1].strip().upper().split()[0]
        send_telegram(f"🔍 {ticker} analiz
ediliyor (trend + erken uyarı)...",
chat_id)
        def do_full_analyze(tk=ticker,
cid=chat_id):
            shared_df = td_get_ohlcvt(tk,
outputsize=210)
            if shared_df is None:
                send_telegram(f"{tk} için
veri alınamadı. Sembolü kontrol et.", cid)
                return

            trend_result =
analyze_stock(tk, df=shared_df)

```

```

        print(f"[DEBUG do_full_analyze]
{tk}: trend_result={'VAR' if trend_result
else 'NONE'}, içerik={trend_result}")
        ew_result =
early_warning_scan(tk, df=shared_df)
        print(f"[DEBUG do_full_analyze]
{tk}: ew_result={'VAR' if ew_result else
'NONE'}")

        trend_signal =
trend_result.get("signal") if trend_result
else None
        trend_info =
trend_result.get("info") if trend_result
else None
        trend_ipo =
trend_result.get("ipo_message") if
trend_result and trend_result.get("ipo")
else None
        ew_signal =
ew_result.get("signal") if ew_result else
None
        ew_info = ew_result.get("info")
if ew_result else None
        ew_ipo =
ew_result.get("ipo_message") if ew_result
and ew_result.get("ipo") else None
        print(f"[DEBUG do_full_analyze]
{tk}: trend_signal={'VAR' if trend_signal
else 'YOK'}, trend_info={'VAR' if
trend_info else 'YOK'}")

        if trend_signal:
            send_telegram(trend_signal,

```

```

cid)
    if ew_signal:
        send_telegram(ew_signal,
cid)

        # Hiçbir gerçek sinyal yoksa,
        bilgi amaçlı durumu göster
        if not trend_signal and not
        ew_signal:
            if trend_ipo:

send_telegram(trend_ipo, cid)
            elif trend_info:

send_telegram(trend_info, cid)
            if ew_ipo and ew_ipo !=
trend_ipo:
                send_telegram(ew_ipo,
cid)
            elif ew_info:
                send_telegram(ew_info,
cid)
            if not trend_ipo and not
trend_info and not ew_ipo and not ew_info:
                send_telegram(f"{tk}
için veri alınamadı. Sembolü kontrol et.",
cid)

        # SİNYAL + HABER KOMBİNASYONU:
        Gerçek sinyal geldiğinde önemli
        # haberler de otomatik
        gönderilir. Sadece önemli kategorilerdeki
        # haberler (kazanç, CEO,
        birleşme vb.) dahil edilir – spam olmaz.

```

```

        if trend_signal or ew_signal:
            time.sleep(1)
            news_items =
fetch_news_for_symbol(tk,
importance_filter=True)
            if news_items:
                header = f"📰 <b>{tk} –
Güncel Önemli Haberler</b>"
                send_telegram(header,
cid)
                time.sleep(0.5)
                for item in
news_items[:3]:
                    nid = get_news_id()
                    title =
item.get('title', '')
                    summary =
item.get('summary', '')
                    date =
item.get('date', '')
                    link =
item.get('link', '')
                    news_archive[nid] =
{'title': title, 'symbol': tk, 'date':
now_tr().strftime('%d.%m.%Y %H:%M')}
                    translated =
translate_and_interpret_news(title,
summary, tk)
                    msg = f"📰 <b>{nid}
– {tk}</b>\n🕒 {date}\n\n{translated}"
                    if link:
                        msg += f"\n🔗
{link}"
                    send_telegram(msg,

```

```

cid)

        time.sleep(1)

threading.Thread(target=do_full_analyze).start()

    return

    # ELLIOTT DALGA SAYIMI (eş değer:
    eliott, eliot, elliot, dalga)
    if any(t.startswith(x) for x in
    ['/elliott', '/eliot', '/elliot', '/dalga']):
        rest =
    re.sub(r'^/(elliott|eliot|dalga)', '',
    t).strip()
        if any(x in rest for x in
    ['tara', 'scan']):
            send_telegram("🌊 Elliott Dalga
    taraması başladı...", chat_id)
            def
    do_elliott_scan(cid=chat_id):
                batch =
    get_trading_pool(20)
                found = 0
                for ticker in batch:
                    result =
    eliott_wave_analysis(ticker)
                    if result and
    result.get("signal"):

    send_telegram(result["signal"], cid)
                found += 1
                time.sleep(2)
                time.sleep(1.2)
            if found == 0:

```

```

        send_telegram("Elliott
Dalga taraması tamamlandı. Tamamlanmış 5
dalga yapısı bulunamadı.", cid)

threading.Thread(target=do_elliott_scan).st
art()

        elif rest:
            ticker = rest.upper().split()
[0]
            send_telegram(f"🌐 {ticker}
Elliott Dalga analizi yapılıyor...",
chat_id)

        def
do_single_elliott(tk=ticker, cid=chat_id):
            result =
elliott_wave_analysis(tk)
            if result and
result.get("signal"):

send_telegram(result["signal"], cid)
            elif result and
result.get("ipo"):

send_telegram(result["ipo_message"], cid)
            elif result and
result.get("info"):

send_telegram(result["info"], cid)
            else:
                send_telegram(f"{tk}
için veri alınamadı.", cid)

threading.Thread(target=do_single_elliott).
start()

```

```

        else:
            send_telegram("Kullanım: /dalga  
[HİSSE] veya /dalga tara", chat_id)
            return

    # FORMASYON SİNYALİ (eş değer:  
formation, formasyon, pattern)
    if any(t.startswith(x) for x in  
['/formation', '/formasyon', '/pattern']):
        rest =  
re.sub(r'^(/(formation|formasyon|pattern)',  
'', t).strip()
        if any(x in rest for x in  
['tara', 'scan']):
            send_telegram("\ud83d\ude09 Formasyon  
taraması başladı...", chat_id)
            def  
do_formation_scan(cid=chat_id):
                batch =  
get_trading_pool(20)  
found = 0  
for ticker in batch:  
    result =  
    formation_analysis(ticker)  
    if result and  
result.get("signal"):  
  
send_telegram(result["signal"], cid)  
        found += 1  
        time.sleep(2)  
time.sleep(1.2)  
if found == 0:  
  
send_telegram("Formasyon taraması
```



```

tamamlandı. Net formasyon bulunamadı.",
cid)

threading.Thread(target=do_formation_scan).
start()
    elif rest:
        ticker = rest.upper().split()
[0]
        send_telegram(f"📊 {ticker}
formasyon analizi yapılıyor...", chat_id)
    def
do_single_formation(tk=ticker,
cid=chat_id):
        result =
formation_analysis(tk)
        if result and
result.get("signal"):

send_telegram(result["signal"], cid)
        elif result and
result.get("ipo"):

send_telegram(result["ipo_message"], cid)
        elif result and
result.get("info"):

send_telegram(result["info"], cid)
        else:
            send_telegram(f"{tk}
için veri alınamadı.", cid)

threading.Thread(target=do_single_formation
).start()
    else:

```

```

        send_telegram("Kullanım:
/formasyon [HİSSE] veya /formasyon tara",
chat_id)
        return

    # HABER
    if any(t.startswith(x) for x in
['/haber', '/news']):
        parts = text.split(None,1)
        if len(parts) < 2:
            send_telegram("Kullanım: /haber
[HİSSE]", chat_id)
            return
        ticker =
parts[1].strip().upper().split()[0]
        send_telegram(f"📰 {ticker}
haberleri aranıyor...", chat_id)
        def do_news(tk=ticker,
cid=chat_id):
            news_items =
fetch_news_for_symbol(tk,
importance_filter=True)
            if not news_items:
                # Önemli haber
bulunamadıysa tüm haberleri dene
                news_items =
fetch_news_for_symbol(tk,
importance_filter=False)
            if not news_items:
                send_telegram(f"{tk}
için güncel haber bulunamadı.", cid)
                return
            send_telegram(f"📌 {tk}
için önemli kategoride haber bulunamadı,
```

```

tüm haberler gösteriliyor.", cid)
    for item in news_items:
        nid = get_news_id()
        title = item.get('title',
'Başlık yok')
        summary =
item.get('summary', '')
        date = item.get('date', '')
        link = item.get('link', '')
        news_archive[nid] =
{'title': title, 'symbol': tk, 'date':
now_tr().strftime('%d.%m.%Y %H:%M')}
        # Claude API ile Türkçe
çeviri ve yorum
        translated =
translate_and_interpret_news(title,
summary, tk)
        msg = f"📰 <b>{nid} - {tk}</b>\n🕒 {date}\n\n{translated}"
        if link:
            msg += f"\n🔗 {link}"
        send_telegram(msg, cid)
        time.sleep(1)

threading.Thread(target=do_news).start()
return

# GÜNÜN EN ÇOK KAZANANLARI
if any(t.startswith(x) for x in
['/kazananlar', '/gainers', '/topkazanan', '/t
op']):
    send_telegram("📈 Günün en çok
kazanan hisseleri getiriliyor...", chat_id)
def do_gainers(cid=chat_id):

```

```

        gainers =
fetch_top_gainers_table(limit=50)
        if not gainers:
            send_telegram("Kazananlar
listesi şu an alınamadı, lütfen tekrar
dene.", cid)
            return

        lines = []
        lines.append(f"📈 <b>GÜNÜN EN
ÇOK KAZANANLARI</b>")
        lines.append(f"🕒
{now_tr().strftime('%d.%m.%Y %H:%M')}")

        lines.append("—————
—")

        lines.append(f"{'Hisse':<6}
{'Kazanç':>7} {'Dolar Hacmi':>12}
{'Adet':>10}")

        lines.append("—————
—")

        for i, g in enumerate(gainers,
1):
            symbol = g['symbol']
            change_pct =
g['change_pct']
            volume = g['volume']
            dollar_vol =
g['dollar_volume']

            # Dolar hacmini kısa
formatta göster

```

```

        if dollar_vol >=
1_000_000_000:
            dvol_str =
f"${dollar_vol/1_000_000_000:.1f}B"
            elif dollar_vol >=
1_000_000:
                dvol_str =
f"${dollar_vol/1_000_000:.1f}M"
            else:
                dvol_str =
f"${dollar_vol/1_000:.0f}K"

        # Adet hacmini kısa
formatta göster
        if volume >= 1_000_000:
            vol_str = f"
{volume/1_000_000:.1f}M"
            elif volume >= 1_000:
                vol_str = f"
{volume/1_000:.0f}K"
            else:
                vol_str = str(volume)

        lines.append(f"{i:>2}.
{symbol:<5} +{change_pct:.1f}%
{dvol_str:>10} {vol_str:>8}")

        # Telegram mesaj limiti (4096)
aşılabilir, ikiye böl
        msg1 = "\n".join(lines[:30])
        msg2_lines = lines[:5] +
lines[30:]
        msg2 = "\n".join(msg2_lines)

```

```

        send_telegram(f"<code>{msg1}
</code>", cid)
        if len(gainers) > 25:
            time.sleep(0.5)
            send_telegram(f"<code>
{msg2}</code>", cid)

threading.Thread(target=do_gainers).start()
return

# ALARMLAR
if any(t.startswith(x) for x in
['/alarm', '/alert']):
    parts = text.split()
    if any(t.startswith(x) for x in
['/alarmlarim', '/alerts']):
        if alerts:
            msg = "🔔
<b>ALARMLARIM</b>\n" + "\n".join(f"{k} →
{v['price']}" for k,v in alerts.items())
            send_telegram(msg, chat_id)
        else:
            send_telegram("Aktif alarm
yok.", chat_id)
        return
    if any(x in t for x in
['sil', 'delete']):
        if len(parts) >= 3:
            ticker = parts[-1].upper()
            if ticker in alerts:
                del alerts[ticker]
                send_telegram(f"✅
{ticker} alarmı silindi.", chat_id)
            else:

```

```

        send_telegram(f"
{ticker} için alarm bulunamadı.", chat_id)
        return
    if len(parts) >= 3:
        try:
            ticker = parts[1].upper()
            price = float(parts[2])
            alerts[ticker] = {'price':
price, 'chat_id': chat_id, 'thread_id':
thread_id}
            send_telegram(f"🔔 Alarm
kuruldu: {ticker} → {price}", chat_id)
        except:
            send_telegram("Kullanım:
/alarm [HİSSE] [FİYAT]", chat_id)
        return

    # PORTFÖY
    if any(t.startswith(x) for x in
['/portfoy', '/portföy', '/portfolio']):
        parts = text.split()
        if any(x in t for x in
['ekle', 'add']):
            if len(parts) >= 5:
                try:
                    ticker =
parts[2].upper()
                    price = float(parts[3])
                    qty = float(parts[4])
                    portfolio[ticker] =
{'price': price, 'qty': qty, 'chat_id':
chat_id, 'thread_id': thread_id}
                    send_telegram(f"✅
{ticker} - {qty} adet @ {price} eklendi.",

```

```

chat_id)

        except:

            send_telegram("Kullanım: /portfoy ekle
[HİSSE] [FİYAT] [ADET]", chat_id)
            return
            if len(parts) >= 2 and not any(x in
t for x in ['ekle','add']):
                ticker = parts[1].upper()
                if ticker in portfolio:
                    p = portfolio[ticker]
                    send_telegram(f"
{ticker}\nAlış: {p['price']} x {p['qty']}
adet", chat_id)
                else:
                    send_telegram(f"{ticker}
portföyde yok.", chat_id)
                return
            if portfolio:
                msg = " <b>PORTFÖYÜM</b>\n" +
"\n".join(f"{k}: {v['price']} x {v['qty']}"
for k,v in portfolio.items())
                send_telegram(msg, chat_id)
            else:
                send_telegram("Portföy boş.",
chat_id)
            return

        # TAKİP
        if any(t.startswith(x) for x in
['/takip','/track']):
            parts = text.split()
            if any(x in t for x in
['takiplerim','mytracks']):

```



```

        if tracked:
            msg = "📌 <b>TAKİP
LISTEM</b>\n" + "\n".join(f"{k} – Giriş:
{v['entry']} Stop:{v['stop']} Hedef:
{v['target']}" for k,v in tracked.items())
            send_telegram(msg, chat_id)
        else:
            send_telegram("Takip
listesi boş.", chat_id)
        return
    if len(parts) >= 5:
        try:
            ticker = parts[1].upper()
            entry, stop, target =
float(parts[2]), float(parts[3]),
float(parts[4])
            tracked[ticker] = {'entry':
entry, 'stop': stop, 'target': target,
'chat_id': chat_id, 'thread_id': thread_id}
            send_telegram(f"📌 {ticker}
takibe alındı.\nGiriş:{entry} Stop:{stop}
Hedef:{target}", chat_id)
        except:
            send_telegram("Kullanım:
/takip [HİSSE] [GİRİŞ] [STOP] [HEDEF]",
chat_id)
        return

    send_telegram("Komut tanınmadı.
/yardim veya /help yazabilirsin.", chat_id)

# =====
# WEBHOOK & ROUTES
# =====

```

```

@app.route("/webhook", methods=["POST"])
def webhook():
    try:
        update =
request.get_json(force=True)
        message = update.get("message", {})
        text = message.get("text", "")
        chat = message.get("chat", {})
        chat_id = str(chat.get("id", ""))
        thread_id =
message.get("message_thread_id")
        if chat_id:
            key = f"{chat_id}_{thread_id}"
if thread_id else chat_id
            known_chats[key] = {
                "chat_id": chat_id,
                "thread_id": thread_id,
                "title": chat.get("title")
or chat.get("first_name") or
chat.get("username") or "(isimsiz)",
                "type": chat.get("type",
""),
                "last_seen":
now_tr().strftime("%d.%m.%Y %H:%M:%S"),
                "last_text": text[:50] if
text else "",
            }

        # Grup ana sohbeti
(HAWK_GROUP_CHAT_ID) içindeki "Genel"
konusundan
        # gelen mesajlar/komutlar tamamen
yok sayılır (bot hiç cevap vermez).
        # Telegram forum gruplarında

```

```

"Genel" konusunun message_thread_id'si
    # yoktur (None gelir), diğer tüm
konularda thread_id dolu olur.
    is_genel_konu = (
        HAWK_GROUP_CHAT_ID
        and chat_id ==
str(HAWK_GROUP_CHAT_ID)
        and thread_id is None
    )
    if text and chat_id and not
is_genel_konu:

threading.Thread(target=handle_command,
args=(text, chat_id, thread_id)).start()
    except:
        pass
    return jsonify({"ok": True})

@app.route("/debug/chats")
def debug_chats():
    """
        Bota şu ana kadar mesaj gönderilmiş
        (herhangi bir mesaj yazılmış) tüm
        chat/grupların ID, başlık ve son
        görülme zamanını listeler. Kanal
        kurulumu sırasında chat ID öğrenmek
        için kullanılır.
    """
    return jsonify(known_chats)

@app.route("/set_webhook")
def set_webhook():
    domain =
os.environ.get("RAILWAY_PUBLIC_DOMAIN", "")

```

```

        if not domain:
            return jsonify({"error":
"RAILWAY_PUBLIC_DOMAIN not set"})
        r =
requests.get(f"https://api.telegram.org/bot
{TELEGRAM_TOKEN}/setWebhook", params=
{"url": f"https://{domain}/webhook"})
        return jsonify(r.json())

@app.route("/")
def home():
    return jsonify({"status": "Hawk Signal
Bot - NASDAQ 🦅", "pool_source":
last_pool_source["source"],
"scraper_available": TV_SCRAPER_AVAILABLE})

@app.route("/test")
def test():
    send_telegram(f"🦅 <b>Hawk Signal Bot
Aktif!</b>\n\n✅ Canlı piyasa taraması\n✅
Erken Uyarı Sistemi (7 kriter)\n✅ Haber
entegrasyonu\n\nTarayıcı kütüphanesi:
{'Aktif ✅' if TV_SCRAPER_AVAILABLE else
'Yedek modda ⚠️'}")
    return jsonify({"status": "Test mesajı
gönderildi!"})

# =====
# OTOMATİK TARAMA
# =====

# =====
# PİYASA SAATLERİ (NASDAQ, DST + Resmi
Tatiller + Uyku Modu)

```

```
# =====
# NASDAQ seansları (ABD Doğu Saati / ET):
#   Pre-market: 04:00 - 09:30 ET
#   Normal seans: 09:30 - 16:00 ET
#   Kapanış sonrası: 16:00 - 20:00 ET
# Bot mantığı: Pre-market başlamadan 2 saat
# önce uyanır, kapanış sonrası
# seans bitince tekrar uyur. Hafta sonu
# (Cumartesi tam gün + Pazar,
# Pazartesi açılışından 2 saat öncesine
# kadar) ve resmi tatil günlerinde
# tamamen kapalı sayılır.

NASDAQ_HOLIDAYS_2026 = {
    (1, 1), (1, 19), (2, 16), (4, 3), (5,
25), (6, 19),
    (7, 3), (9, 7), (11, 26), (12, 25),
}

def get_us_eastern_now():
    """
    ABD Doğu Saati'ni (ET) DST'yi otomatik
    hesaba katarak döndürür.
    DST kuralı: Mart'ın 2. Pazarı'ndan
    Kasım'ın 1. Pazarı'na kadar EDT (UTC-4),
    diğer zamanlarda EST (UTC-5).
    """
    utc_now = datetime.now(timezone.utc)
    year = utc_now.year

    # Mart'ın 2. Pazarı'nı bul
    march1 = datetime(year, 3, 1,
tzinfo=timezone.utc)
    days_to_sunday = (6 - march1.weekday())
```

```

% 7
    first_sunday_march = march1 +
timedelta(days=days_to_sunday)
    dst_start = first_sunday_march +
timedelta(days=7) # 2. Pazar

    # Kasım'ın 1. Pazarı'nı bul
    nov1 = datetime(year, 11, 1,
tzinfo=timezone.utc)
    days_to_sunday_nov = (6 -
nov1.weekday()) % 7
    dst_end = nov1 +
timedelta(days=days_to_sunday_nov)

    is_dst = dst_start <= utc_now < dst_end
    offset_hours = -4 if is_dst else -5
    et_tz =
timezone(timedelta(hours=offset_hours))
    return utc_now.astimezone(et_tz)

def is_market_holiday(et_dt):
    return (et_dt.month, et_dt.day) in
NASDAQ_HOLIDAYS_2026

def get_market_status():
    """
    Botun şu an hangi modda olması
gerektiğini döndürür:
    'active' - bot uyanık, tarama yapmalı
    'sleep' - bot uykuda, tarama yapmamalı
    Ayrıca insan-okunabilir bir açıklama
metni ve session_type döner.
    session_type: 'early_wake' |
'premarket' | 'regular' | 'afterhours' |

```

```

'sleep'
    Tarama sıklığı bu session_type'a göre
    belirlenir (bkz.
    get_scan_interval_seconds).
    """
    et = get_us_eastern_now()
    weekday = et.weekday() # 0=Pazartesi
    ... 5=Cumartesi, 6=Pazar
    et_time = et.hour * 60 + et.minute #
    dakika cinsinden

    pre_market_start = 4 * 60          # 04:00
    ET
    market_open = 9 * 60 + 30          # 09:30
    ET
    market_close = 16 * 60             # 16:00
    ET
    after_hours_end = 20 * 60          # 20:00
    ET
    wake_up_time = pre_market_start - 2 *
    60 # Pre-market'ten 2 saat önce = 02:00 ET

    if is_market_holiday(et):
        return "sleep", f"Resmi tatil
        ({et.strftime('%d.%m.%Y')}), piyasa
        kapalı", "sleep"

    if weekday == 5: # Cumartesi
        return "sleep", "Cumartesi, piyasa
        tamamen kapalı", "sleep"

    if weekday == 6: # Pazar
        return "sleep", "Pazar, piyasa
        tamamen kapalı", "sleep"

```

```
        if weekday == 0: # Pazartesi – gece
yarisından wake_up_time'a kadar hâlâ uykuda
            if et_time < wake_up_time:
                return "sleep", "Pazartesi,
açılış öncesi uyku modu sürüyor", "sleep"

        if weekday == 4: # Cuma – after_hours
bitince hafta sonu uykusu başlar
            if et_time >= after_hours_end:
                return "sleep", "Cuma kapanış
sonrası seans bitti, hafta sonu uykusu",
"sleep"

        # Pazartesi-Cuma arası, normal gün içi
mantık
        if et_time < wake_up_time:
            return "sleep", "Gece uyku modu
(pre-market'ten 2 saat öncesine kadar)",
"sleep"
        if wake_up_time <= et_time <
pre_market_start:
            return "active", "Açılış öncesi
uyanma penceresi (pre-market'e 2 saat
var)", "early_wake"
        if pre_market_start <= et_time <
market_open:
            return "active", "Pre-market
seansı", "premarket"
        if market_open <= et_time <
market_close:
            return "active", "Normal seans
(piyasa açık)", "regular"
        if market_close <= et_time <
```



```

after_hours_end:
    return "active", "Kapanış sonrası seans", "afterhours"
    return "sleep", "Kapanış sonrası seans bitti, uyku modu", "sleep"

def get_scan_interval_seconds():
    """
    Tarama sıklığını session_type'a göre belirler:
    - early_wake (açılışa 2 saat var): 30 dakika
    - premarket: 30 dakika
    - regular (normal seans): 30 dakika
    - afterhours (kapanış sonrası): 30 dakika
    - sleep: bir sonraki kontrol için 10 dakika (zaten tarama yapılmaz, sadece tekrar uyanma kontrolü)
    """
    _, _, session_type = get_market_status()
    if session_type == "sleep":
        return 10 * 60
    else: # early_wake, premarket, regular, afterhours – artık hepsi aynı
        return 30 * 60

def is_nasdaq_hours():
    status, _, _ = get_market_status()
    return status == "active"

def is_bot_active():
    return bot_manual_state["active"] and

```

```

is_nasdaq_hours()

def auto_scan_loop():
    time.sleep(15)
    send_telegram(f"🦅 <b>Hawk Signal Bot –  
NASDAQ</b>\n\n✅ Canlı piyasa taraması\n✅  
Trend + Erken Uyarı sistemleri aktif\n✅  
Seans içi ve kapanış sonrası 30 dk aralıkla  
otomatik tarama\n\nTarayıcı kütüphanesi:  
{ 'Aktif ✅' if TV_SCRAPER_AVAILABLE else  
'Yedek modda ⚠️' }, HAWK_GROUP_CHAT_ID,  
THREAD_ID_SISTEM)

    while True:
        try:
            if is_bot_active():
                reset_sector_tracking()
                batch =
get_trading_pool(30)

                if
last_pool_source["source"] == "memory":
                    send_kanal("⚠️ Otomatik  
tarama: canlı veri alınamadı, hafızadaki  
son başarılı taramaya geçildi.", "sistem")
                elif
last_pool_source["source"] == "none":
                    send_kanal("❌ Otomatik  
tarama: hiçbir veri kaynağına erişilemedi,  
bu döngü atlandı.", "sistem")
                    time.sleep(1500)
                    continue

                for ticker in batch:

```

```
        result =
analyze_stock(ticker)
        if result and
result.get("signal"):

send_kanal(result["signal"], "trend")

send_news_for_signal(ticker, "Trend
Sinyali")

        time.sleep(2)
        time.sleep(0.5)

        ew_batch =
random.sample(batch, min(10, len(batch)))
        for ticker in ew_batch:
            result =
early_warning_scan(ticker)
            if result and
result.get("signal"):

send_kanal(result["signal"], "erkenuyari")

send_news_for_signal(ticker, "Erken Uyarı")
            time.sleep(2)
            time.sleep(0.5)

        # Elliott Dalga ve
Formasyon taraması: kredi/worker yükünü
        # korumak için daha küçük
bir alt-küme üzerinde çalışır.
        pattern_batch =
random.sample(batch, min(5, len(batch)))
        for ticker in
pattern_batch:
```

```

        result =
elliott_wave_analysis(ticker)
        if result and
result.get("signal"):

send_kanal(result["signal"], "elliott")

send_news_for_signal(ticker, "Elliott
Dalga")

        time.sleep(2)
        time.sleep(0.5)

    for ticker in
pattern_batch:
        result =
formation_analysis(ticker)
        if result and
result.get("signal"):

send_kanal(result["signal"], "formasyon")

send_news_for_signal(ticker, "Formasyon")
        time.sleep(2)
        time.sleep(0.5)

    for ticker, data in
list(tracked.items()):
        try:
            df =
td_get_ohlc(ticker, 5)
            if df is not None:
                current =
float(df['Close'].iloc[-1])
                if current <=

```

```

data['stop']:

send_telegram(f"🛑 <b>STOP – {ticker}</b>\n${current:.2f} → Stop
${data['stop']}", data['chat_id'],
data.get('thread_id'))
                                elif current >=
data['target']:

send_telegram(f"🎯 <b>HEDEF – {ticker}</b>\n${current:.2f} → Hedef
${data['target']}", data['chat_id'],
data.get('thread_id'))
                                except:
                                    pass

                                for ticker, data in
list(alerts.items()):
                                try:
                                    df =
td_get_ohlcv(ticker, 5)
                                    if df is not None:
                                        current =
float(df['Close'].iloc[-1])
                                        if current >=
data['price']:

send_telegram(f"🔔 <b>ALARM – {ticker}</b>\n${current:.2f} → Hedef
${data['price']}", data['chat_id'],
data.get('thread_id'))
                                del
alerts[ticker]
                                except:

```

```
                pass
            except:
                pass

time.sleep(get_scan_interval_seconds())

def news_scan_loop():
    """
        Kanal 5 (Haberler) için BAĞIMSIZ
        periyodik tarama. Teknik sinyal
        üretilsin üretilmesin çalışır – amaç
        önemli haberi hiç kaçırmamak.
        İki kaynağı tarar:
        1. Finnhub genel piyasa haberleri
        (category=general)
        2. Ham havuzdaki (örneklemesiz, TÜM
        canlı mover'lar) her sembol için
        Finnhub company-news
        Aynı haber (Finnhub id) daha önce
        gönderildiyse tekrar gönderilmez –
        bu sayede sinyal tetiklemeyle zaten
        atılan bir haber burada
        mükerrer gelmez.
    """
    time.sleep(90) # diğer döngülerle
    çakışmasın diye biraz geriden başlasın
    while True:
        try:
            interval =
get_scan_interval_seconds()
            if interval != 10 * 60: #
sleep modunda değilsek (piyasa tamamen
kapalı değilse)
```

```

# 1) Piyasa geneli haberler
general_items =
fetch_finnhub_general_news(importance_filte
r=True, max_items=15)
for item in general_items:
    nid = item["id"]
    if
is_news_already_sent(nid):
        continue
    related =
item.get("related", "")
    label =
related.split(",")[0].strip() if related
else "Genel Piyasa"
    translated =
translate_and_interpret_news(item["title"],
item["summary"], label)
    msg = f"📰 <b>{label} –
Piyasa Haberi</b>\n{translated}"
    if item.get("link"):
        msg += f"\n🔗
{item['link']}"
    send_kanal(msg,
"haber")

    mark_news_sent(nid)
    time.sleep(1)

# 2) Ham havuzdaki
(örneklemesiz) semboller için önemli
haberler
raw_pool =
get_raw_trading_pool()
for sym in raw_pool:
    items =

```

```

fetch_news_for_symbol(sym,
importance_filter=True, days_back=1)
    for item in items:
        nid = item["id"]
        if
is_news_already_sent(nid):
            continue
        translated =
translate_and_interpret_news(item["title"],
item["summary"], sym)
        msg = f"📰 <b>{sym}
– Önemli Haber</b>\n{translated}"
        if
item.get("link"):
            msg += f"\n🔗
{item['link']}"
            send_kanal(msg,
"haber")
            mark_news_sent(nid)
            time.sleep(1) #
Finnhub free plan 60 istek/dk limitine
takılmamak için
        except Exception as e:
            print(f"[DEBUG news_scan_loop]
EXCEPTION: {e}")
            time.sleep(30 * 60)

threading.Thread(target=auto_scan_loop,
daemon=True).start()
threading.Thread(target=elliott_auto_scan_l
oop, daemon=True).start()
threading.Thread(target=news_scan_loop,
daemon=True).start()

```



```
if __name__ == "__main__":  
    port = int(os.environ.get("PORT",  
5000))  
    app.run(host="0.0.0.0", port=port)
```